

курс «Глубокое обучение»

Эффективное обучение нейронных сетей **часть 1**

Александр Дьяконов

План: борьба с переобучением / сложностью

Очень много параметров $\Rightarrow$ переобучение

- Нормировки (Normalization of Data)
- Инициализация весов
- Верификация – ранний останов (Early Stopping)
- Настройка темпа обучения (Learning Rate)
- Мини-батчи (Mini-Batches) / Batch-обучение
- Продвинутая оптимизация
- Регуляризация + Weight Decay
- Зашумление (inject noise)
- Max-norm-регуляризация
- Обрезка градиентов (Gradient clipping)
- Dropout
- Батч-нормализация

Нормировки (Data Normalization)

Признак $X = (X_1, \dots, X_m)$

$$\mu = \frac{1}{m} \sum_{i=1}^m X_i \text{ - среднее признака}$$

Стандартизация

$$\hat{X} = \frac{X - \mu}{\sigma}$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (X_i - \mu)^2 \text{ - дисперсия признака}$$

иногда м.б. PCA



Минутка кода: нормировки

```
# простое (но не очень хорошее) вычисление статистик
loader = DataLoader(train_set, batch_size=len(train_set), num_workers=1)
data = next(iter(loader))
data[0].mean(), data[0].std()

(tensor(0.2860), tensor(0.3530))

# новый нормированный датасет
train_set_normal = torchvision.datasets.FashionMNIST(root='./data',
                                                       train=True,
                                                       download=True,
                                                       transform=transforms.Compose([transforms.ToTensor(),
                                                       transforms.Normalize(mean, std)]))
```

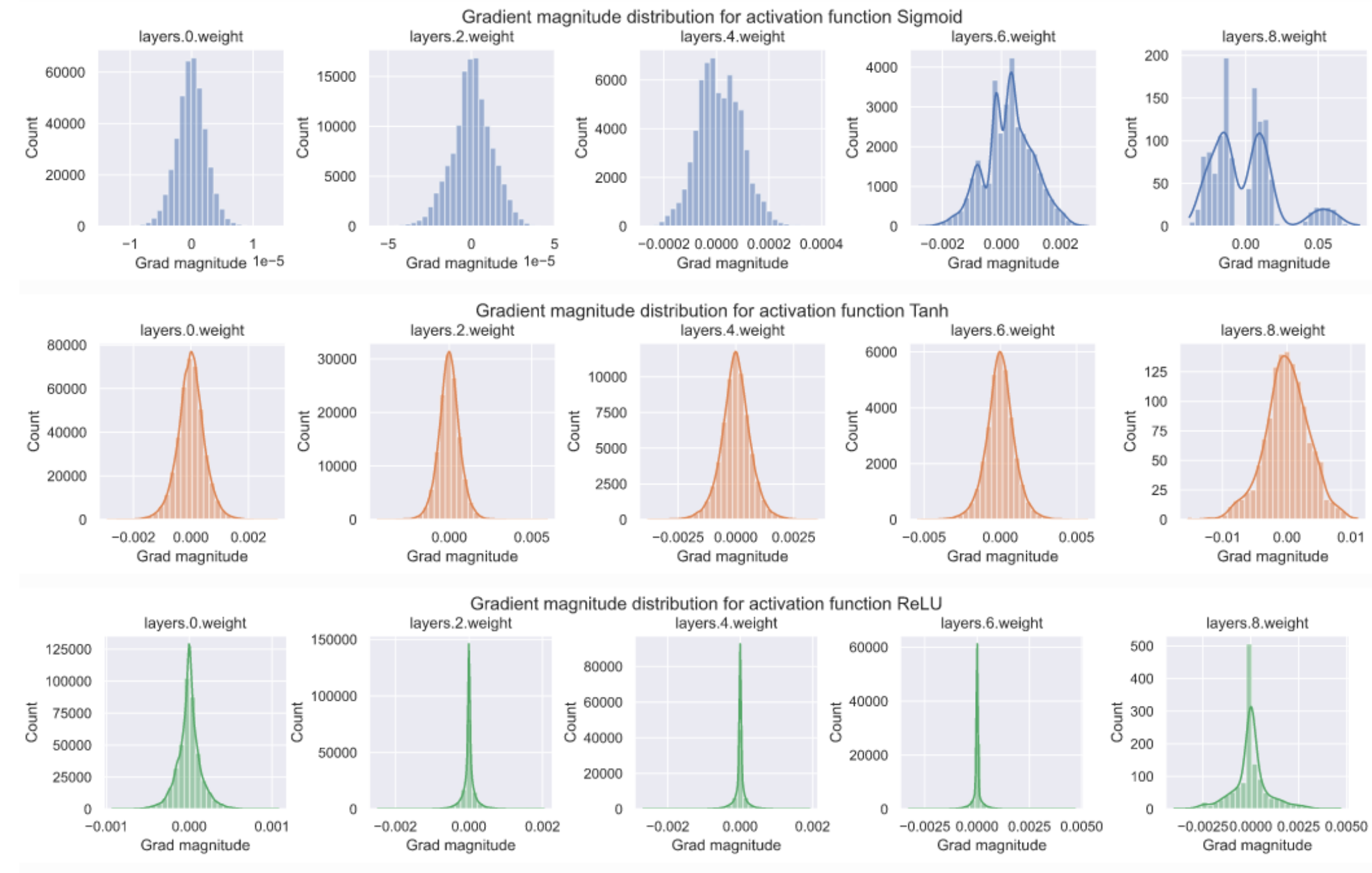
<https://deeplizard.com/learn/video/lu7TCu7HeYc>

Инициализация весов: ключевые идеи

- нарушение симметричности
(чтобы нейроны были разные)
 - недопущение «насыщенности» нейрона
(почти всегда близок к нулю или 1)
 - входы на все слои должны иметь одинаковую дисперсию
(для избегания «насыщения» нейронов)
это показатель, что сигнал проходит и не затухает / увеличивается
 - градиенты на всех слоях должны иметь одинаковую дисперсию
тогда подберём темп обучения сразу для всех слоёв
- смещения := 0 (зависит от того, где смещение; если на выходе...)

Поиграйтесь! – <https://deeplearning.ai/ai-notes/initialization/>

Распределения и дисперсии в нейронных сетях



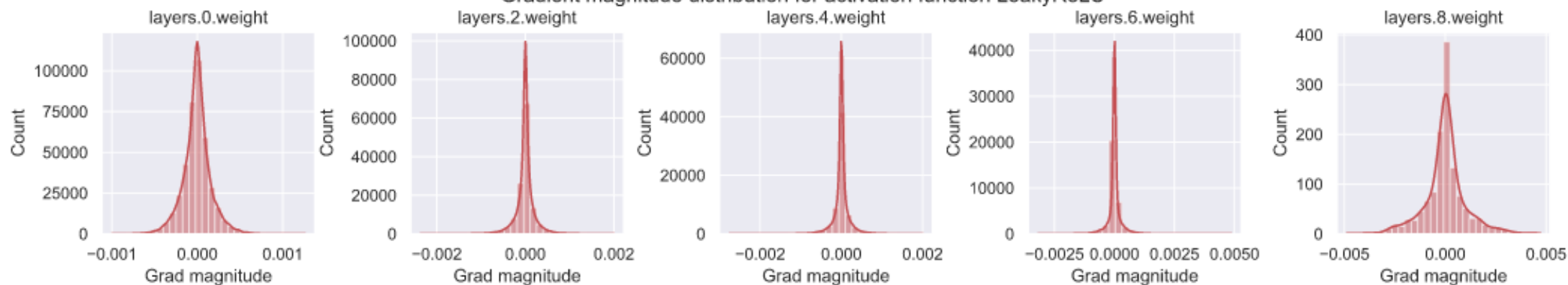
У сигмоиды очень большие градиенты на выходе – плохо!

И маленькие на входе.

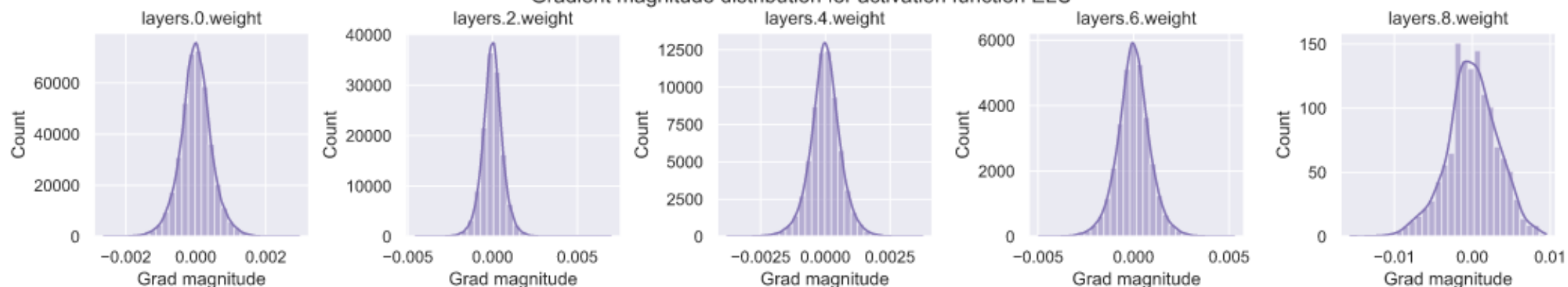
У ReLU пик в нуле – ПОНЯТНО

Распределения и дисперсии в нейронных сетях

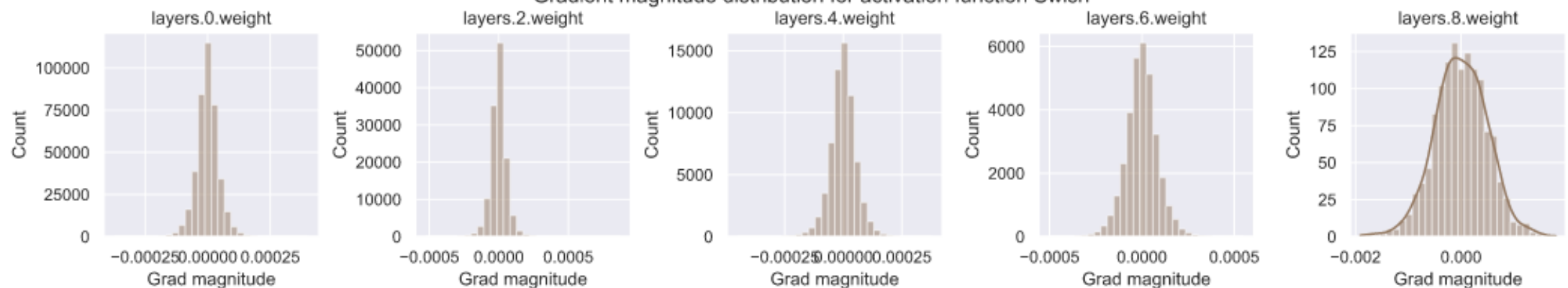
Gradient magnitude distribution for activation function LeakyReLU



Gradient magnitude distribution for activation function ELU



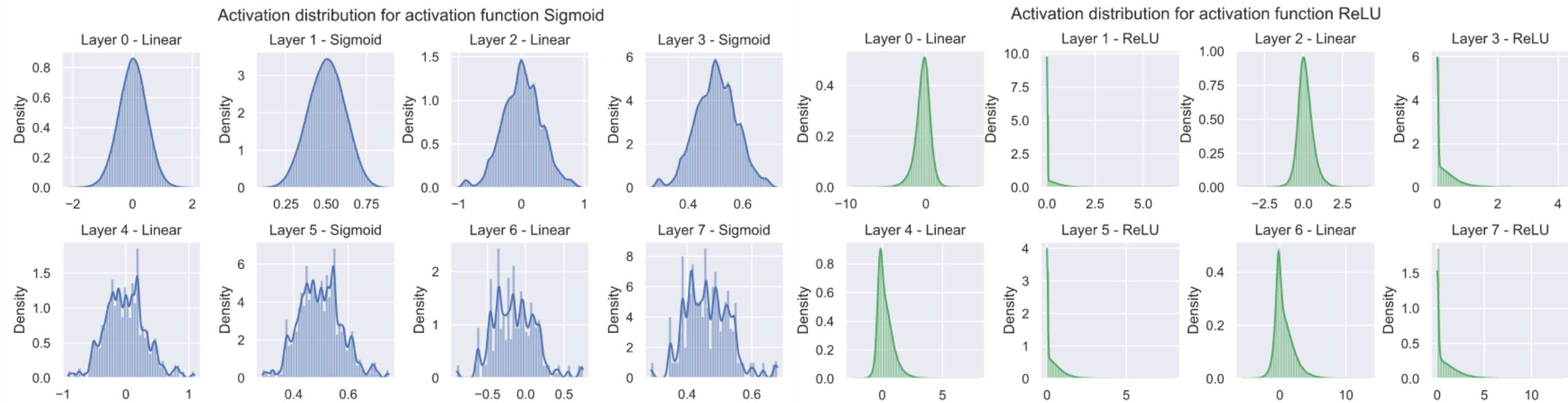
Gradient magnitude distribution for activation function Swish



Кроме сигмoиды
плохих эффектов
нет

Но тут важно,
какая была
инициализация!

Распределения и дисперсии в нейронных сетях



Здесь будут разные картинки для разных функций активаций
(при этом качество НС может быть схожим)

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial3/Activation_Functions.html

Xavier (Glorot) initialization

Uniform

$$w_{ij}^{(k)} \sim U \left[-\sqrt{\frac{6}{n_{\text{in}}^{(k)} + n_{\text{out}}^{(k)}}}, +\sqrt{\frac{6}{n_{\text{in}}^{(k)} + n_{\text{out}}^{(k)}}} \right]$$

Normal

$$w_{ij}^{(k)} \sim \text{norm} \left(0, \frac{2}{n_{\text{in}}^{(k)} + n_{\text{out}}^{(k)}} \right)$$

Формула выведена в предположении, что w – i.i.d., z – i.i.d, нет нелинейностей:

$$z^{(k+1)} = f(W^{(k)} z^{(k)}) \equiv W^{(k)} z^{(k)}$$

выводится, что дисперсия должна быть

$$Dw_{ij}^{(k)} = \frac{2}{n_{\text{in}}^{(k)} + n_{\text{out}}^{(k)}}$$

МОЖНО ПОСЧИТАТЬ СМ.

https://www.youtube.com/watch?v=PjS2y8LBMLc&list=PLrCZzMib1e9oOGNLh6_d65HyfdqIJwTQP&index=5

Glorot, Xavier; Bengio, Yoshua (2010-03-31). "Understanding the difficulty of training deep feedforward neural networks". Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings: 249–256.

Kaiming (He) uniform initialization

$$w_{ij}^{(k)} \sim U \left[-\sqrt{\frac{3}{n_{\text{in}}^{(k)}}}, +\sqrt{\frac{3}{n_{\text{in}}^{(k)}}} \right]$$

$$w_{ij}^{(k)} \sim \text{norm} \left(0, 2 / n_{\text{in}}^{(k)} \right)$$

т.к. предыдущее распределение не подходит для ReLU –

He, Kaiming; Zhang, Xiangyu; Ren, Shaoqing; Sun, Jian (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification".

Инициализация весов: код

```
w = torch.empty(3, 5)

nn.init.uniform_(w, a=0.0, b=1.0)

nn.init.normal_(w, mean=0.0, std=1.0)

nn.init.constant_(w, val=0.3)

nn.init.eye_(w)

nn.init.xavier_uniform_(tensor, gain=1.0)

gain = nn.init.calculate_gain('leaky_relu', 0.2)

# по умолчанию в Pytorch (опт для tanh)
nn.init.kaiming_uniform_(w, a=0, mode='fan_in', nonlinearity='leaky_relu')

nn.init.orthogonal_(w, gain=1)

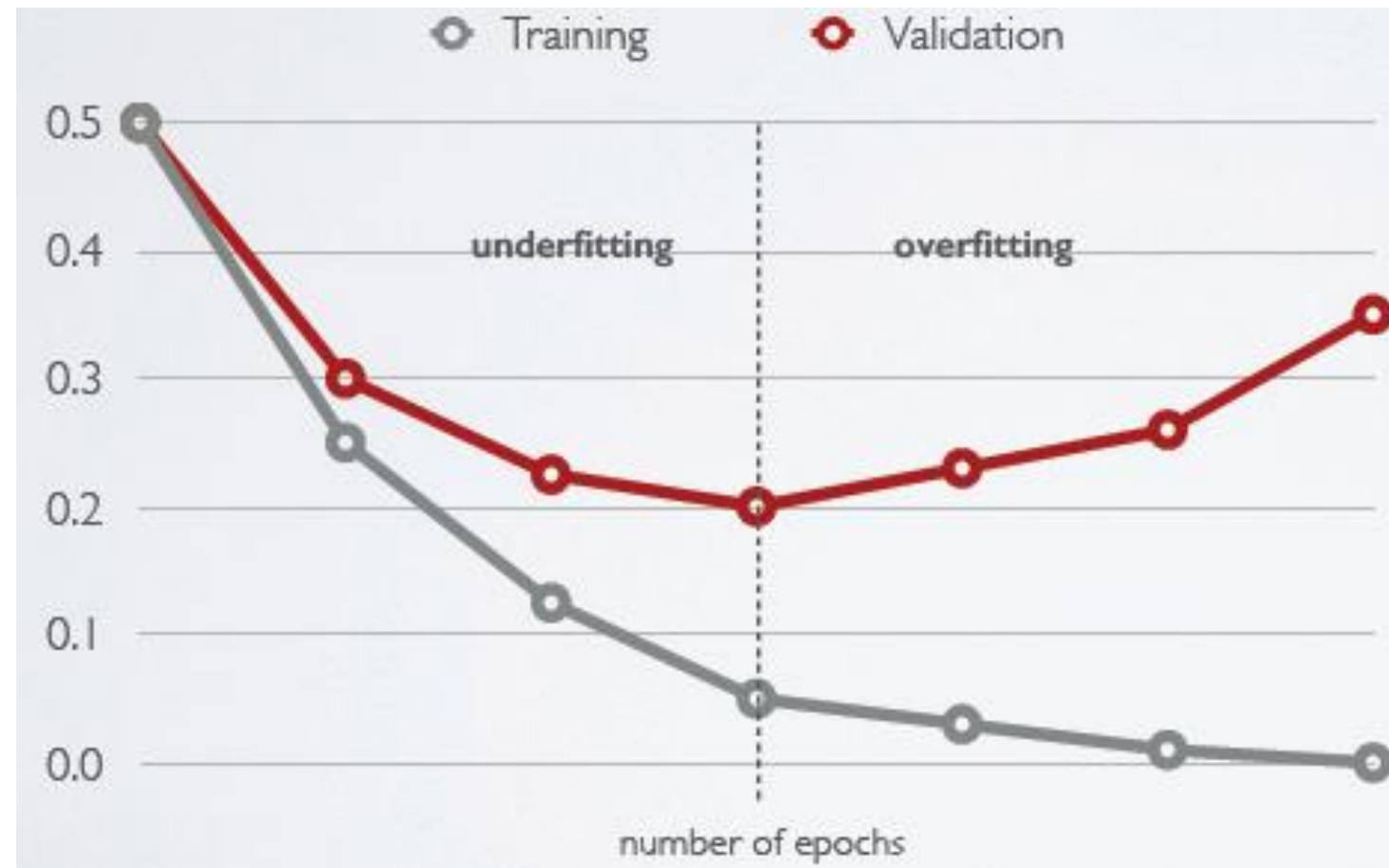
nn.init.sparse_(w, sparsity, std=0.01)
```

$$w_{ij}^{(k)} \sim U[-a, +a]$$

$$a = \text{gain} \cdot \sqrt{\frac{6}{n_{\text{in}}^{(k)} + n_{\text{out}}^{(k)}}}$$

Верификация – ранний останов (Early Stopping)

Смотрим ошибку на отложенной выборке!
Выбираем итерацию, на которой наименьшая ошибка.



Настройка темпа обучения (Learning Rate)

Мини-батчи (mini-batches) / Batch-обучение

$$w^{(t+1)} = w^{(t)} - \frac{\eta}{|I|} \sum_{i \in I} \nabla [l(a(x_i | w^{(t)}), y_i) + \lambda R(w^{(t)})]$$

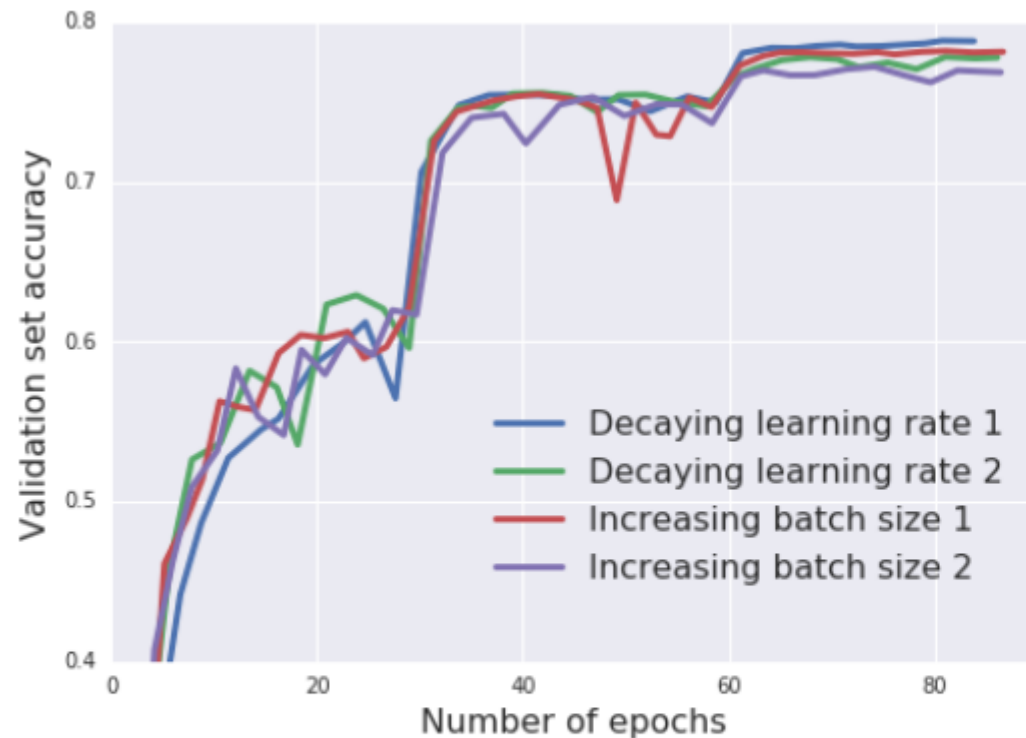
- Градиенты не такие случайные (оцениваем по подвыборке)
- Можно вычислять быстрее (на современных архитектурах)
Можно делать максимальный батч, который влезает в память,
лучше – при котором максимальное ускорение обучения (см. дальше)

$$y_i = \varphi_k(W_k \cdot \dots \cdot \varphi_2(W_2 \cdot \varphi_1(W_1 \cdot x_i)))$$

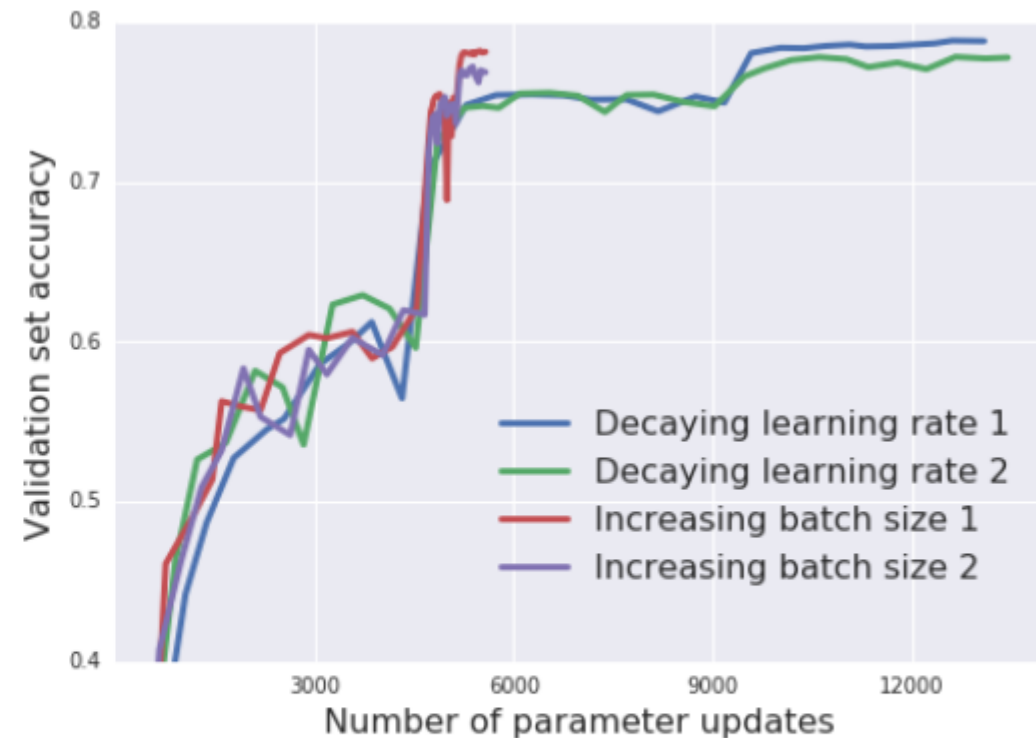
$$[y_{i_1}, \dots, y_{i_k}] = \varphi_k(W_k \cdot \dots \cdot \varphi_2(W_2 \cdot \varphi_1(W_1 \cdot [x_{i_1}, \dots, x_{i_k}])))$$

- Можно делать нормировку по батчу (дальше)
- Можно специально организовывать батчи (ех: должны содержать представителей всех классов)
- Немного противоречит теории / рекомендациям
- Появляется новый гиперпараметр – размер батча (связан с другими гиперпараметрами)

Мини-батчи (mini-batches) / Batch-обучение



(a)



(b)

Figure 6: Inception-ResNet-V2 on ImageNet. Increasing the batch size during training achieves similar results to decaying the learning rate, but it reduces the number of parameter updates from just over 14000 to below 6000. We run each experiment twice to illustrate the variance.

увеличение размера батча – тот же эффект, что и уменьшение темпа обучения
при больших батчах может учиться медленнее!

Мини-батчи: размер батча

Есть мнение, что оптимальный размер батча на [1, 32]

Yann LeCun's facebook:

Training with large minibatches is bad for your health. More importantly, it's bad for your test error. Friends dont let friends use minibatches larger than 32. Let's face it: the *only* people have switched to minibatch sizes larger than one since 2012 is because GPUs are inefficient for batch sizes smaller than 32. That's a terrible reason. It just means our hardware sucks.

Dominic Masters, Carlo Luschi, «Revisiting Small Batch Training for Deep Neural Networks»
<https://arxiv.org/abs/1804.07612>

Плоские минимумы

Считается, что SGD, в отличие от GD, лучше находит «плоские минимумы»
(более надёжно будет и на тесте)

Chiyuan Zhang «Theory of Deep Learning III: Generalization Properties of SGD»
<https://cbmm.mit.edu/sites/default/files/publications/CBMM-Memo-067.pdf>

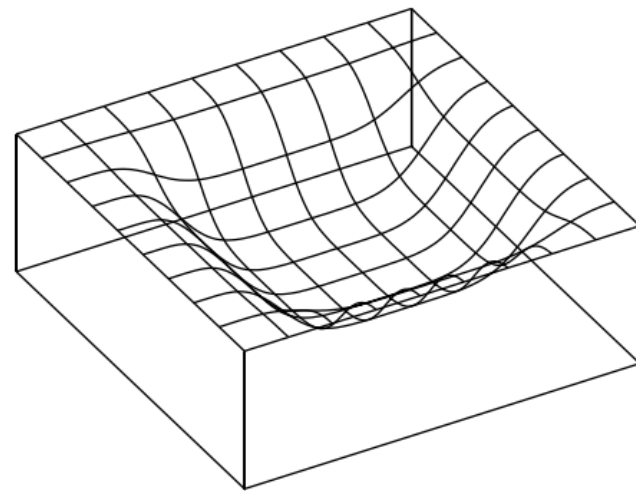


Figure 1: *Example of a “flat” minimum.*

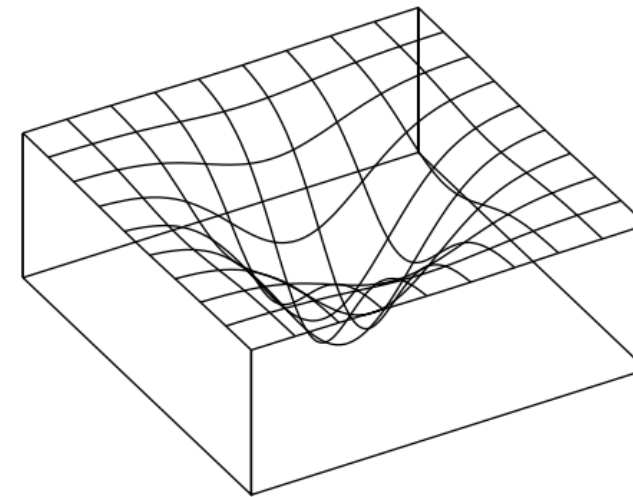


Figure 2: *Example of a “sharp” minimum.*

<https://www.inference.vc/everything-that-works-works-because-its-bayesian-2/>

Размер батча и другие приёмы

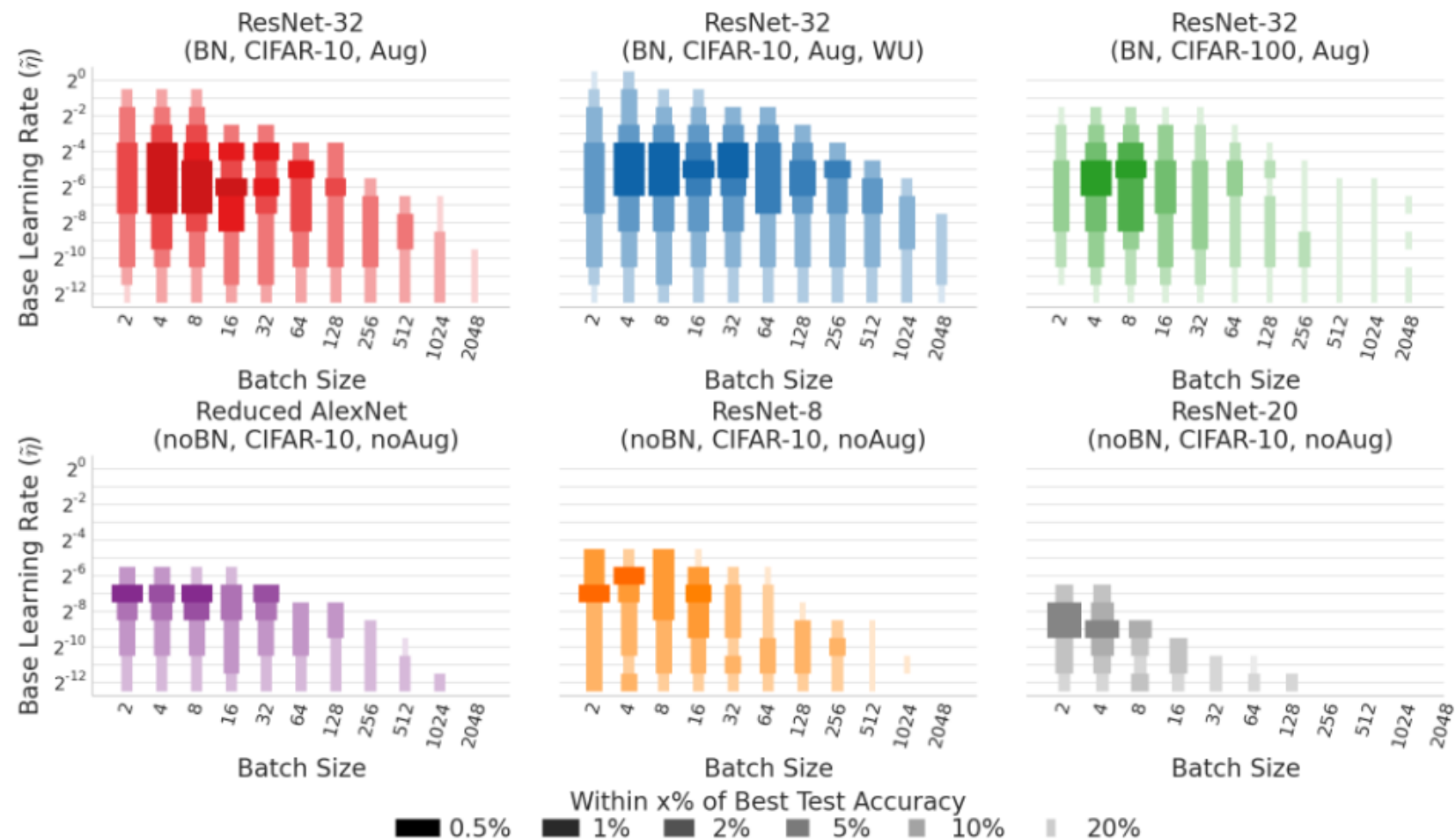


Figure 16: Summary of range of base learning rates $\tilde{\eta} = \eta/m$ that provide reliable convergence, for different network architectures. CIFAR-10 and CIFAR-100 datasets. (BN, noBN: with/without BN; Aug, noAug: with/without data augmentation; WU: with gradual warmup.)

лучше всего batch_size от 2 до 32, использование BN увеличивает оптимальный bs

<https://arxiv.org/pdf/1804.07612.pdf>

Продвинутая оптимизация

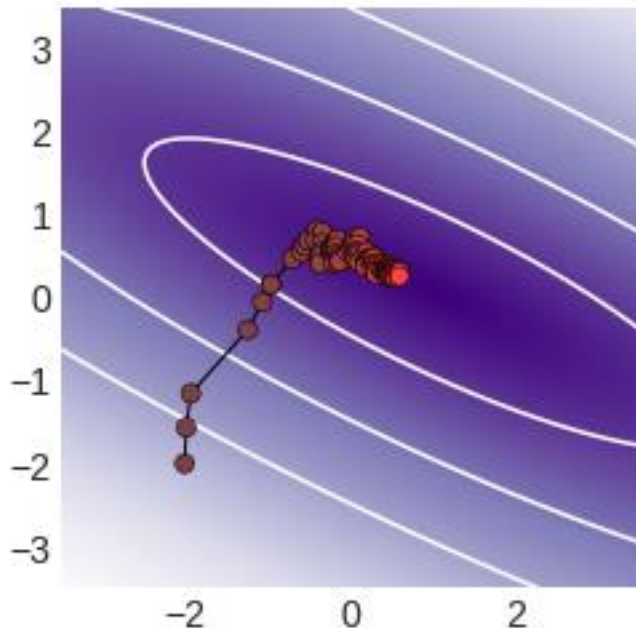
Рассмотрим стохастический градиент

$$w^{(t+1)} = w^{(t)} - \eta \nabla L^{(t)}(w^{(t)})$$

Эпоха – проход по всей обучающей выборке

Итерация – обработка одного батча

Теоретически SGD – выбор случайного объекта, практически – проход по перестановке



- надо случайно перемешивать данные перед каждой эпохой
- уровни функции могут быть сильно вытянуты – adam (дальше)
 - все параметры разные – скорость обучения для каждого параметра
- градиент случаен (зависит от батча) – добавляем градиенту инертность

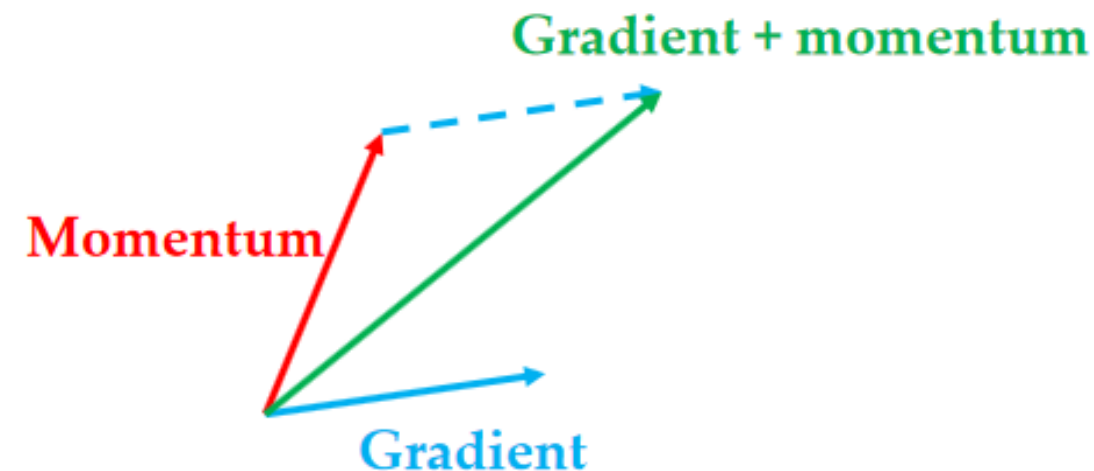
Продвинутая оптимизация: инерция

стохастический градиент с инерцией
(momentum)

$$m^{(t+1)} = \rho m^{(t)} + \nabla L^{(t)}(w^{(t)})$$

$$w^{(t+1)} = w^{(t)} - \eta m^{(t+1)}$$

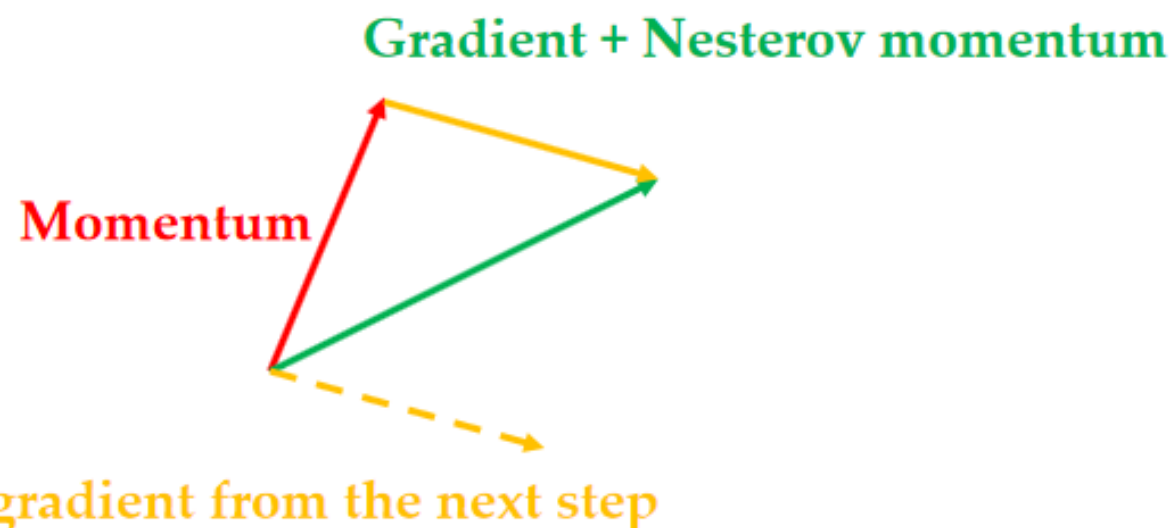
добавление инерции



метод Нестерова

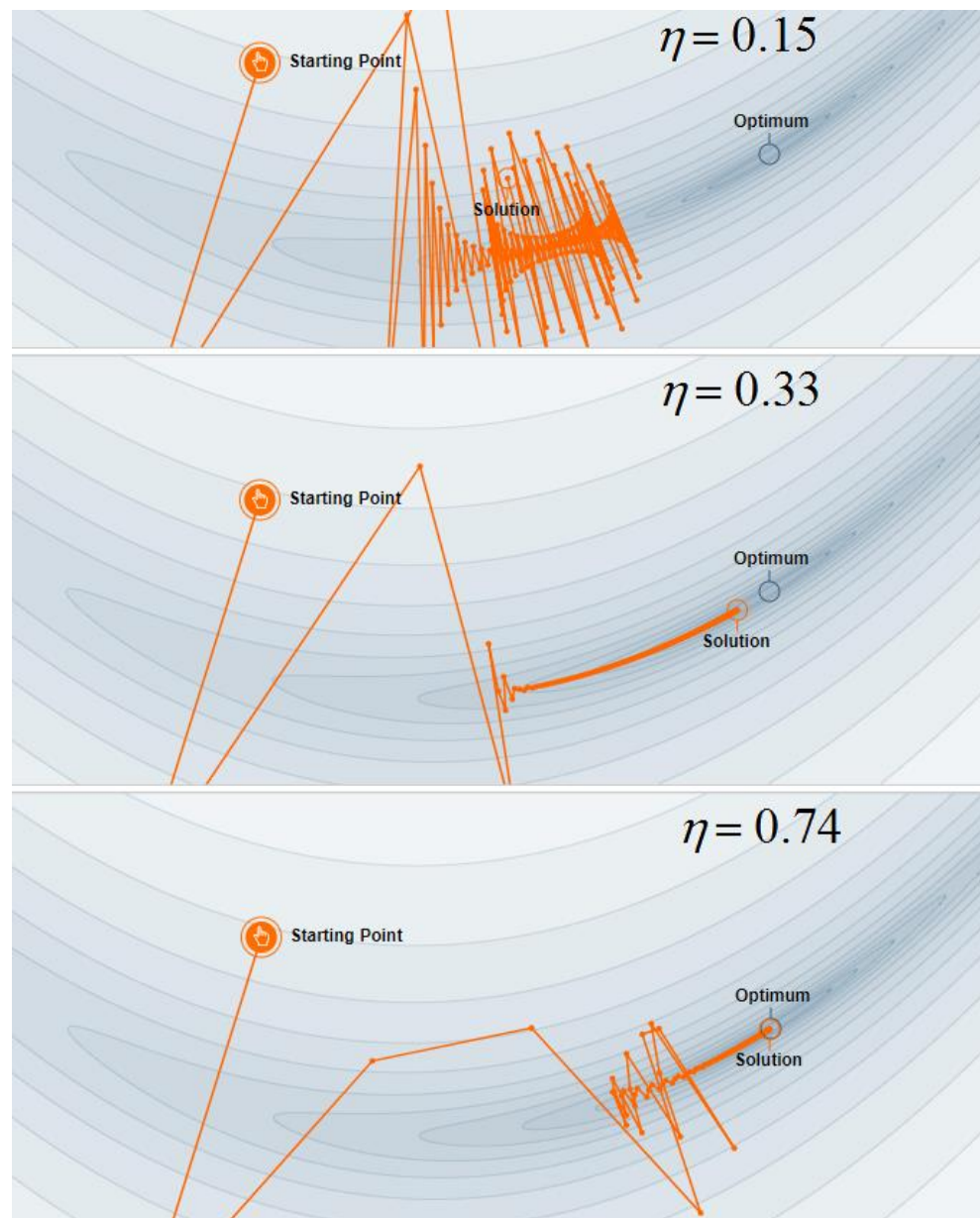
$$m^{(t+1)} = \rho m^{(t)} + \nabla L^{(t)}(w^{(t)} - \eta m^{(t)})$$

$$w^{(t+1)} = w^{(t)} - \eta m^{(t+1)}$$



<https://uvadlc.github.io/lectures/dec2020/lecture3.1.pdf>

Иллюстрация SG с инерцией



Добавление инерции может помочь, когда

- линии уровня вытянуты...
- для проскакивания седловых точек

<https://distill.pub/2017/momentum/>

Продвинутая оптимизация – Адаптивная

Adagrad [Duchi и др., 2011]

$$v_i^{(t+1)} = v_i^{(t)} + (\nabla_i L^{(t)}(w^{(t)}))^2$$
$$w_i^{(t+1)} = w_i^{(t)} - \frac{\eta}{\sqrt{v_i^{(t+1)} + \varepsilon}} \nabla_i L^{(t)}(w^{(t)})$$

RMSprop = «Root Mean Squared Propagation» = Adagrad + MA [Hinton, 2012]

$$v_i^{(t+1)} = \beta v_i^{(t)} + (1 - \beta)(\nabla_i L^{(t)}(w^{(t)}))^2$$
$$w_i^{(t+1)} = w_i^{(t)} - \frac{\eta}{\sqrt{v_i^{(t+1)} + \varepsilon}} \nabla_i L^{(t)}(w^{(t)})$$

Продвинутая оптимизация – Адаптивная

Adam = «Adaptive Moment Estimation» = RMSprop + momentum + correction bias

$$m_i^{(t+1)} = \alpha m_i^{(t)} + (1 - \alpha) \nabla_i L^{(t)}(w^{(t)})$$

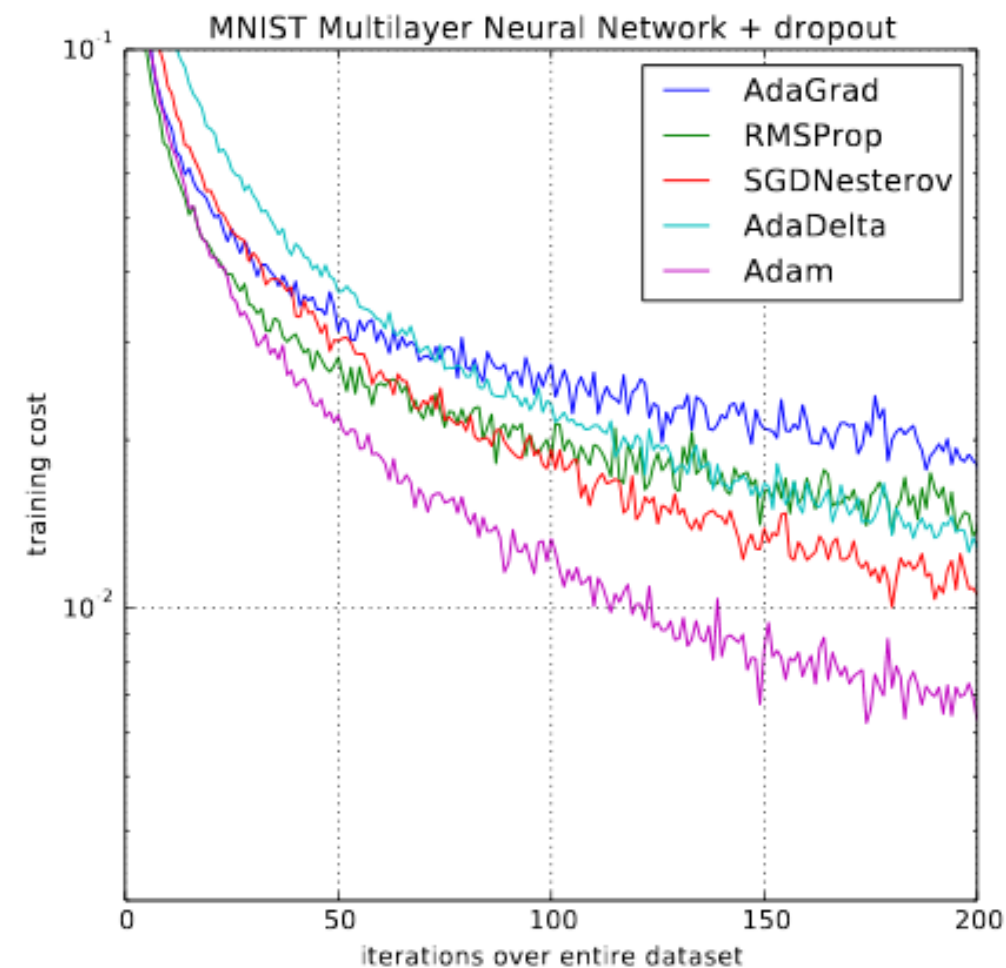
$$v_i^{(t+1)} = \beta v_i^{(t)} + (1 - \beta) (\nabla_i L^{(t)}(w^{(t)}))^2$$

корректировка смещения:

$$\hat{m}_i^{(t+1)} = m_i^{(t+1)} / (1 - \alpha^t)$$

$$\hat{v}_i^{(t+1)} = v_i^{(t+1)} / (1 - \beta^t)$$

$$w_i^{(t+1)} = w_i^{(t)} - \frac{\eta}{\sqrt{\hat{v}_i^{(t+1)} + \epsilon}} \hat{m}_i^{(t+1)}$$



[Kingma, Ba, 2014 <https://arxiv.org/abs/1412.6980>]
226k ссылок, неверное доказательство сходимости

Продвинутая оптимизация

AdaMax [Kingma and Ba, 2015]

ВМЕСТО

$$v_i^{(t+1)} = \beta v_i^{(t)} + (1 - \beta)(\nabla_i L^{(t)}(w^{(t)}))^2$$

$$\hat{v}_i^{(t+1)} = v_i^{(t+1)} / (1 - \beta^t)$$

БУДЕТ

$$v_i^{(t+1)} = \max[\beta v_i^{(t)}, |\nabla_i L^{(t)}(w^{(t)})|]$$

AdaDelta (модификация Adagrad)

$$v_i^{(t+1)} = v_i^{(t)} + (\nabla_i L^{(t)}(w^{(t)}))^2$$

$$\Delta w_i^{(t+1)} = -\frac{\sqrt{\eta_i^{(t)} + \varepsilon}}{\sqrt{v_i^{(t+1)} + \varepsilon}} \nabla_i L^{(t)}(w^{(t)})$$

$$w_i^{(t+1)} = w_i^{(t)} + \Delta w_i^{(t+1)}$$

$$\eta_i^{(t+1)} = \gamma \eta_i^{(t)} + (1 - \gamma)(\Delta w_i^{(t+1)})^2$$

AdamW

$$w_i^{(t+1)} = w_i^{(t)} - \eta \left(\frac{1}{\sqrt{\hat{v}_i^{(t+1)} + \varepsilon}} \hat{m}_i^{(t)} + \alpha_{i,t} w_i^{(t)} \right)$$

немного другая реализация weight decay (**см. ниже**)
в методе Adam

Продвинутая оптимизация

Подбор гиперпараметров очень важен!

Подбор метода оптимизации очень важен!

Хорошие обзоры

«**An overview of gradient descent optimization algorithms**»

<https://runder.io/optimizing-gradient-descent/>

«**An updated overview of recent gradient descent algorithms**»

<https://johnchenresearch.github.io/demon/>

Rectified Adam (RAdam, 2019)

<https://medium.com/@lessw/new-state-of-the-art-ai-optimizer-rectified-adam-radam-5d854730807b>

Продвинутая оптимизация – выбор оптимизатора

Optimizer	State Memory [bytes]	# of Tunable Parameters	Strengths	Weaknesses
SGD	0	1	Often best generalization (after extensive training)	Prone to saddle points or local minima Sensitive to initialization and choice of the learning rate α
SGD with Momentum	$4n$	2	Accelerates in directions of steady descent Overcomes weaknesses of simple SGD	Sensitive to initialization of the learning rate α and momentum β
AdaGrad	$\sim 4n$	1	Works well on data with sparse features Automatically decays learning rate	Generalizes worse, converges to sharp minima Gradients may vanish due to aggressive scaling
RMSprop	$\sim 4n$	3	Works well on data with sparse features Built in Momentum	Generalizes worse, converges to sharp minima
Adam	$\sim 8n$	3	Works well on data with sparse features Good default settings Automatically decays learning rate α	Generalizes worse, converges to sharp minima Requires a lot of memory for the state
AdamW	$\sim 8n$	3	Improves on Adam in terms of generalization Broader basin of optimal hyperparameters	Requires a lot of memory for the state
LARS	$\sim 4n$	3	Works well on large batches (up to 32k) Counteracts vanishing and exploding gradients Built in Momentum	Computing norm of gradient for each layer can be inefficient

<https://www.lightly.ai/post/which-optimizer-should-i-use-for-my-machine-learning-project>

Продвинутая оптимизация: код

```
from torch.utils.data import DataLoader, TensorDataset

X_dataset = TensorDataset(torch.tensor(X, dtype=torch.float32).to(device),
                           torch.tensor(Y, dtype=torch.float32).to(device))
X_dataloader = DataLoader(X_dataset, batch_size=512, shuffle=True)

optimizer = torch.optim.Adam(model.parameters(),
                              lr=0.001,
                              betas=(0.9, 0.999),
                              eps=1e-08,
                              weight_decay=0,
                              amsgrad=False)

for input, target in X_dataset: # тут правда мы игнорируем dataloader:)
    optimizer.zero_grad()
    output = model(input)
    loss = loss_fn(output, target)
    loss.backward()
    optimizer.step()
```

Что выбрать на практике

1) посмотрите, что использовали авторы нейросетевой архитектуры

2) Adam почти всегда хорош
считается даже «over-optimize»

3) Если нет – попробуйте SGD + momentum
можно попробовать разные методы

torch.optim довольно консервативен

Зашумление градиента (Gradient Noise)

Например,
добавление к градиенту шума
(вываливаемся из седловых
точек)

$$g^{(t)} \leftarrow g^{(t)} + \text{norm}\left(0, \frac{\eta}{(1+t)^\gamma}\right)$$

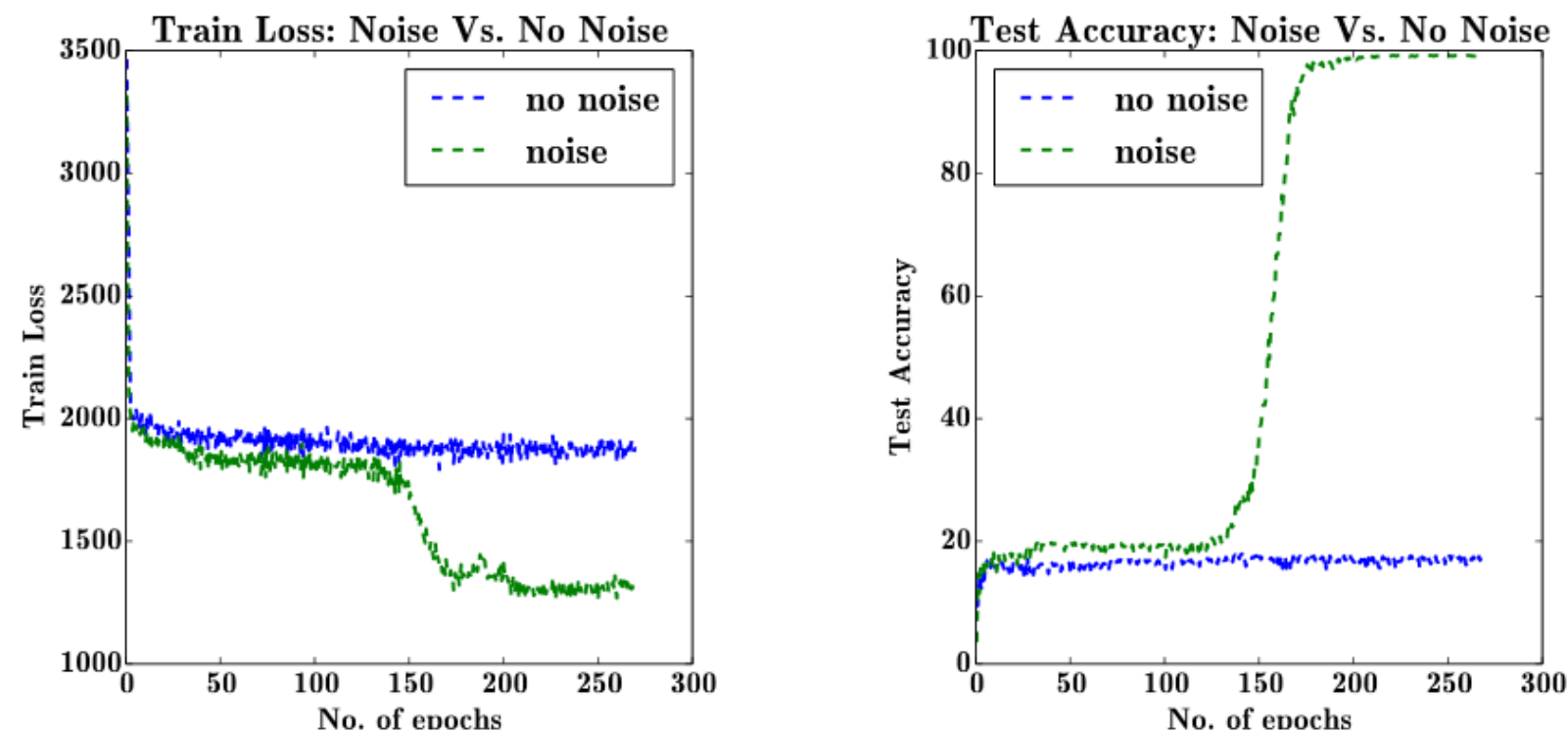


Figure 2: Noise vs. No Noise in our experiment with tables containing 5 columns. The models trained with noise generalizes almost always better.

Neelakantan et al. «Adding gradient noise improves learning for very deep networks» // <https://arxiv.org/abs/1511.06807>

Anandkumar and Ge «Efficient approaches for escaping higher order saddle points in non-convex optimization» <https://arxiv.org/abs/1602.05908>

Регуляризация + Weight Decay

Не применяется к весам к константным входам (смещениям)

L2, L1 – регуляризация

Уменьшение весов (вид регуляризации)

$$w^{(t+1)} = (1 - \lambda)w^{(t)} - \eta \nabla L^{(t)}(w^{(t)})$$

На самом деле это L2-регуляризация (в SGD):

$$\nabla(L(w) + \lambda \|w\|^2) = \nabla L(w) + 2\lambda w$$

$$w^{(t+1)} = w^{(t)} - \eta(\nabla L(w^{(t)}) + 2\lambda w^{(t)}) = (1 - 2\lambda\eta)w^{(t)} - \eta\nabla L(w^{(t)})$$

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1,  
                             momentum=0.9, weight_decay=0)
```

Max-norm-регуляризация

Для каждого нейрона ограничиваем норму весов:

$$\| w_{\text{neuron}} \| \leq c$$

если превысила – проекция

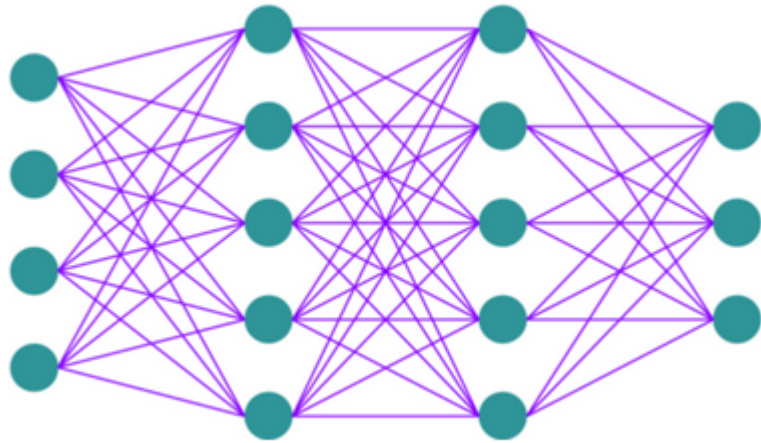
Results from MNIST (handwritten digit recognition)

Method	Unit Type	Architecture	Error %
Standard Neural Net (Simard et al., 2003)	Logistic	2 layers, 800 units	1.60
SVM Gaussian kernel	NA	NA	1.40
Dropout NN	Logistic	3 layers, 1024 units	1.35
Dropout NN	ReLU	3 layers, 1024 units	1.25
Dropout NN + max-norm constraint	ReLU	3 layers, 1024 units	1.06
Dropout NN + max-norm constraint	ReLU	3 layers, 2048 units	1.04
Dropout NN + max-norm constraint	ReLU	2 layers, 4096 units	1.01
Dropout NN + max-norm constraint	ReLU	2 layers, 8192 units	0.95
Dropout NN + max-norm constraint (Goodfellow et al., 2013)	Maxout	2 layers, (5 × 240) units	0.94

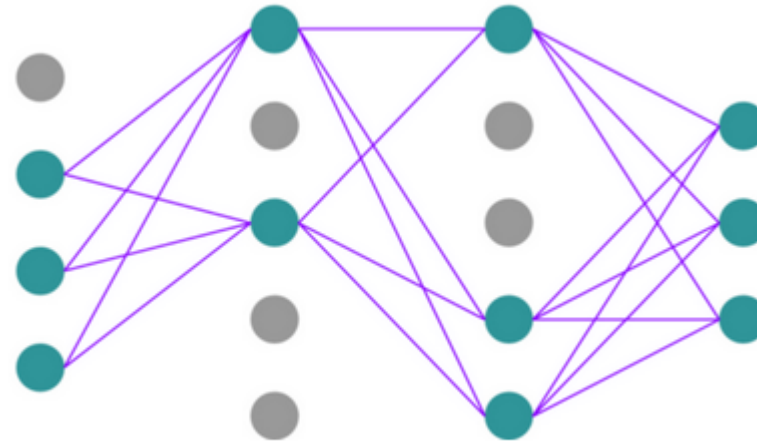
<http://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf>

(Standard) Dropout

Without Dropout



Standard Dropout

Dropout with $p=0.5$

обучение

$$y = f(Wx) \circ m$$

$$m \sim \text{Bernoulli}(1 - p)$$

тест

$$y = (1 - p) f(Wx)$$

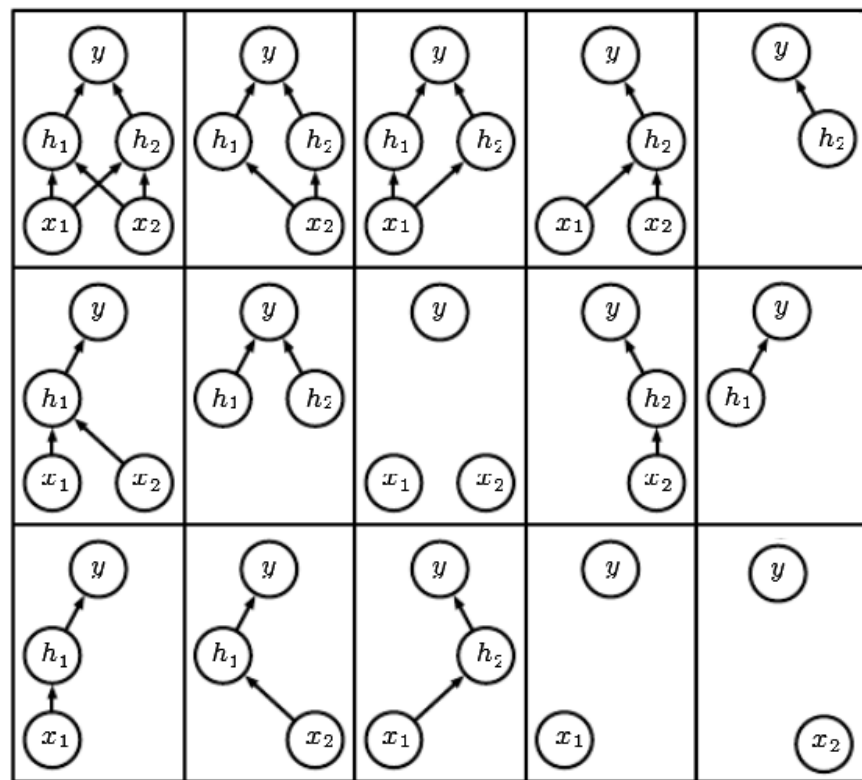
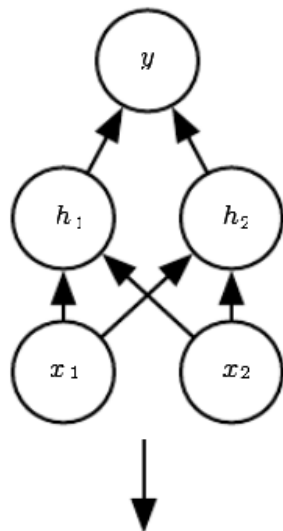
«отключают в режиме теста»

случайное обнуление активаций
 ~ выбрасывание нейронов из сети с вероятностью p
 Обычно в полносвязных слоях, до последнего!

Бонус: оценка уверенности в ответе (а не только регуляризация)
 также оценивают возможность удаления нейронов (Model Compression)

<https://towardsdatascience.com/12-main-dropout-methods-mathematical-and-visual-explanation-58cdc2112293>

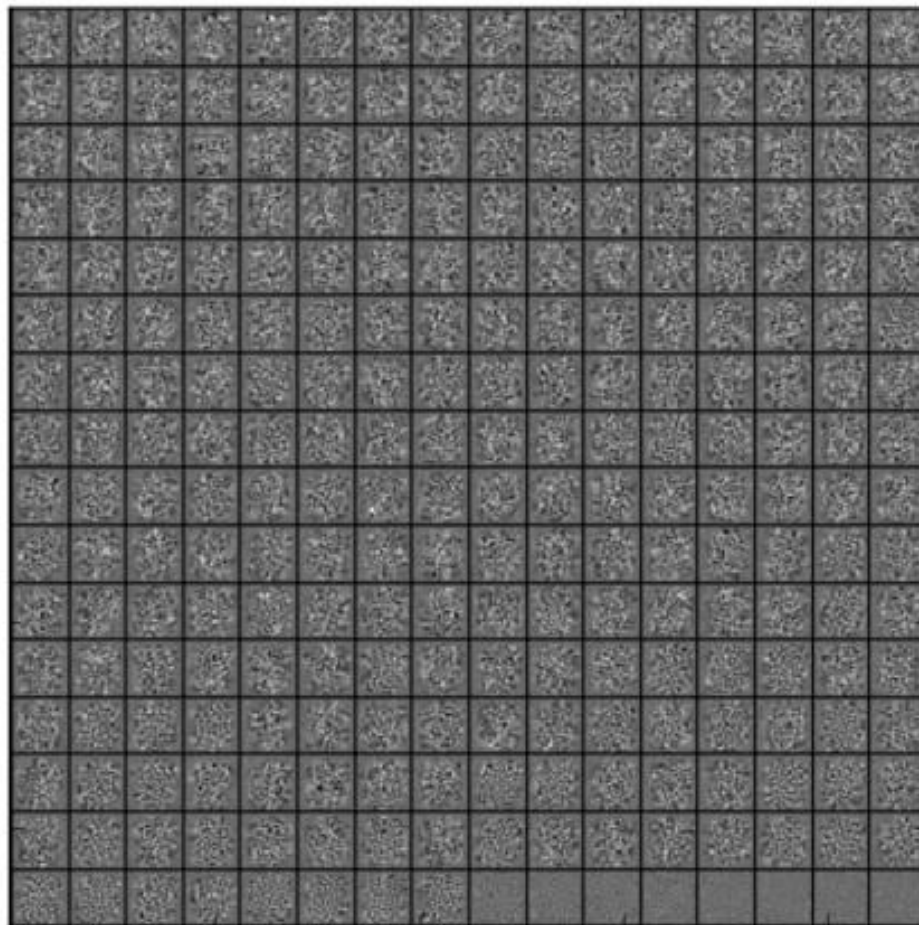
Dropout: обоснования



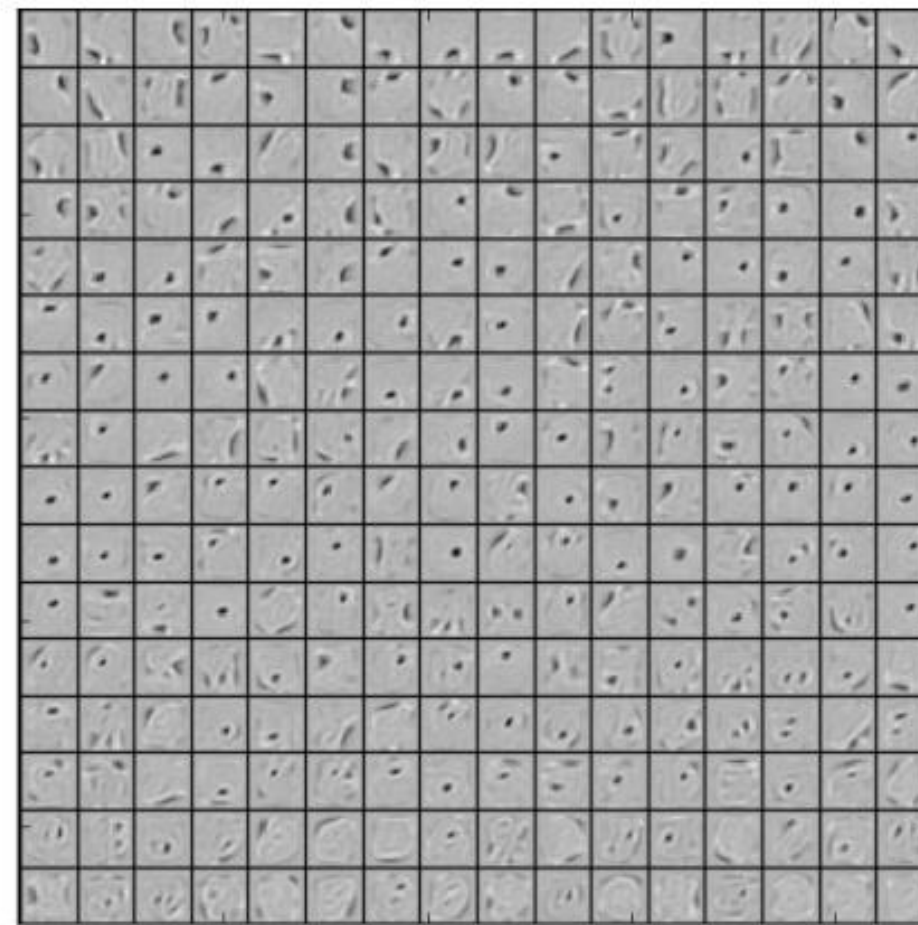
- аналогия с ансамблированием
- байесовский подход
- связь с регуляризацией / аугментацией / теорией информации
- уменьшение co-adaptations / co-dependencies соседних нейронов
- большая робастность нейронов

на рисунке: в отличие от бэгинга все модели делят параметры (не независимы) [DLbook]

Dropout



(a) Without dropout



(b) Dropout with $p = 0.5$.

MNIST: автокодировщик с 1 скрытым слоем и 256 линейными нейронами
[Srivastava, 2014]

Inverted Dropout

обучение

$$y = \frac{1}{1-p} f(Wx) \circ m$$

Тест

$$y = f(Wx)$$

Во время тестирования ничего не делаем...

Inverted Dropout: минутка кода (переключение режимов)

```
optimizer = optim.Adam(model_scaled.parameters())
criterion = nn.CrossEntropyLoss()

num_epochs = 100
for epoch in range(num_epochs):
    model.train()

    optimizer.zero_grad()
    outputs = model(X_train)

    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

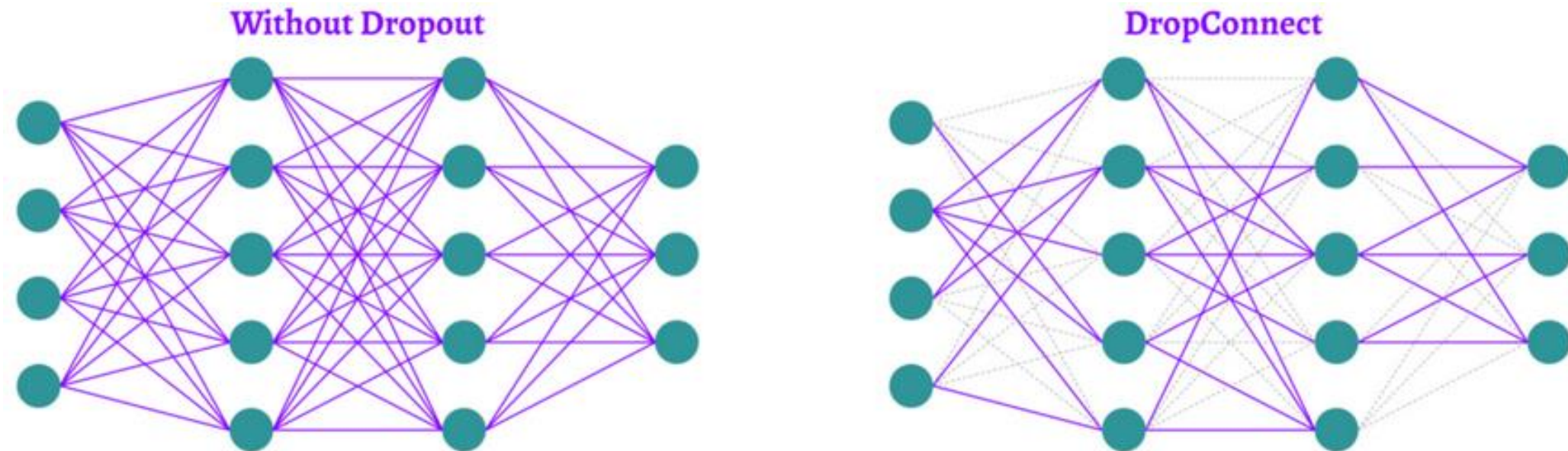
    if (epoch+1) % 10 == 0:
        model.eval()
        with torch.no_grad():
            outputs = model_scaled(X_test_tensor_scaled)
            _, predicted = torch.max(outputs.data, 1)
            correct = (predicted == y_test_tensor).sum().item()
            print(f" Epoch={epoch}, Loss={loss.item():.4f},
                  Accuracy= {correct / len(y_test_tensor):.4f}")
```

Standout

$$m \sim \text{Bernoulli}(g(W_s x))$$

L. J. Ba and B. Frey, Adaptive dropout for training deep neural networks // <https://papers.nips.cc/paper/5032-adaptive-dropout-for-training-deep-neural-networks.pdf>

DropConnect



Зануление отдельных весов, а не нейронов

$$y = f((W \circ M)x)$$

<https://cs.nyu.edu/~wanli/dropc/>

<https://towardsdatascience.com/12-main-dropout-methods-mathematical-and-visual-explanation-58cdc2112293>

DropConnect

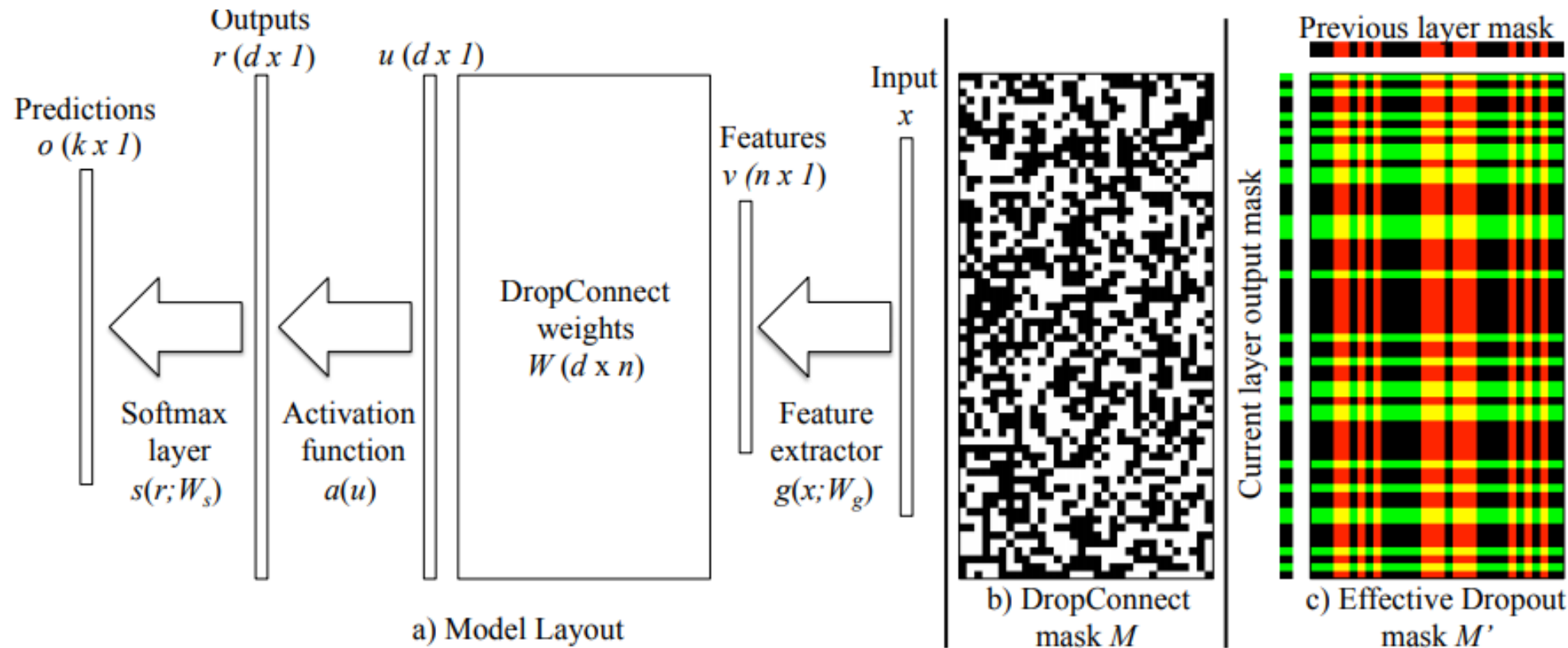
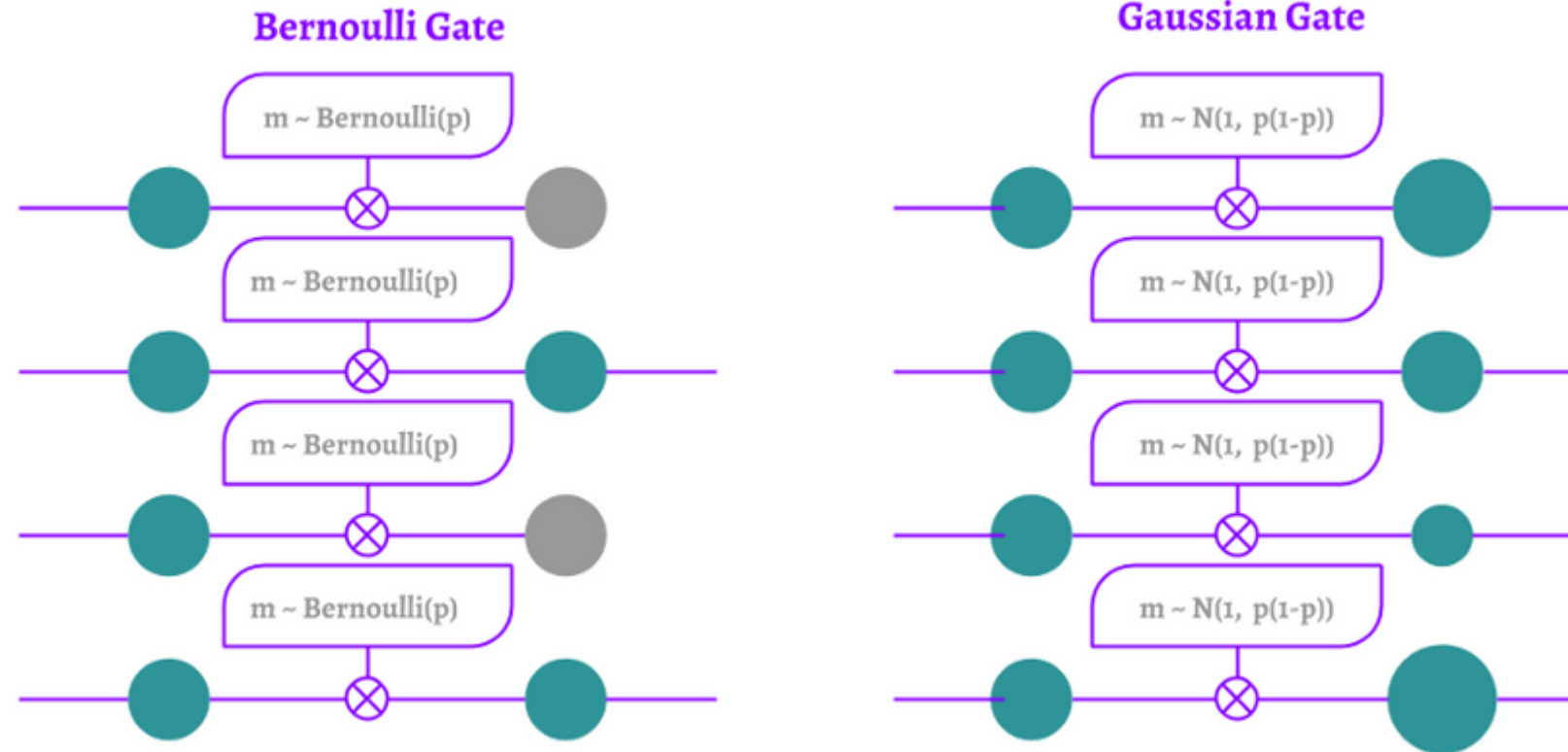


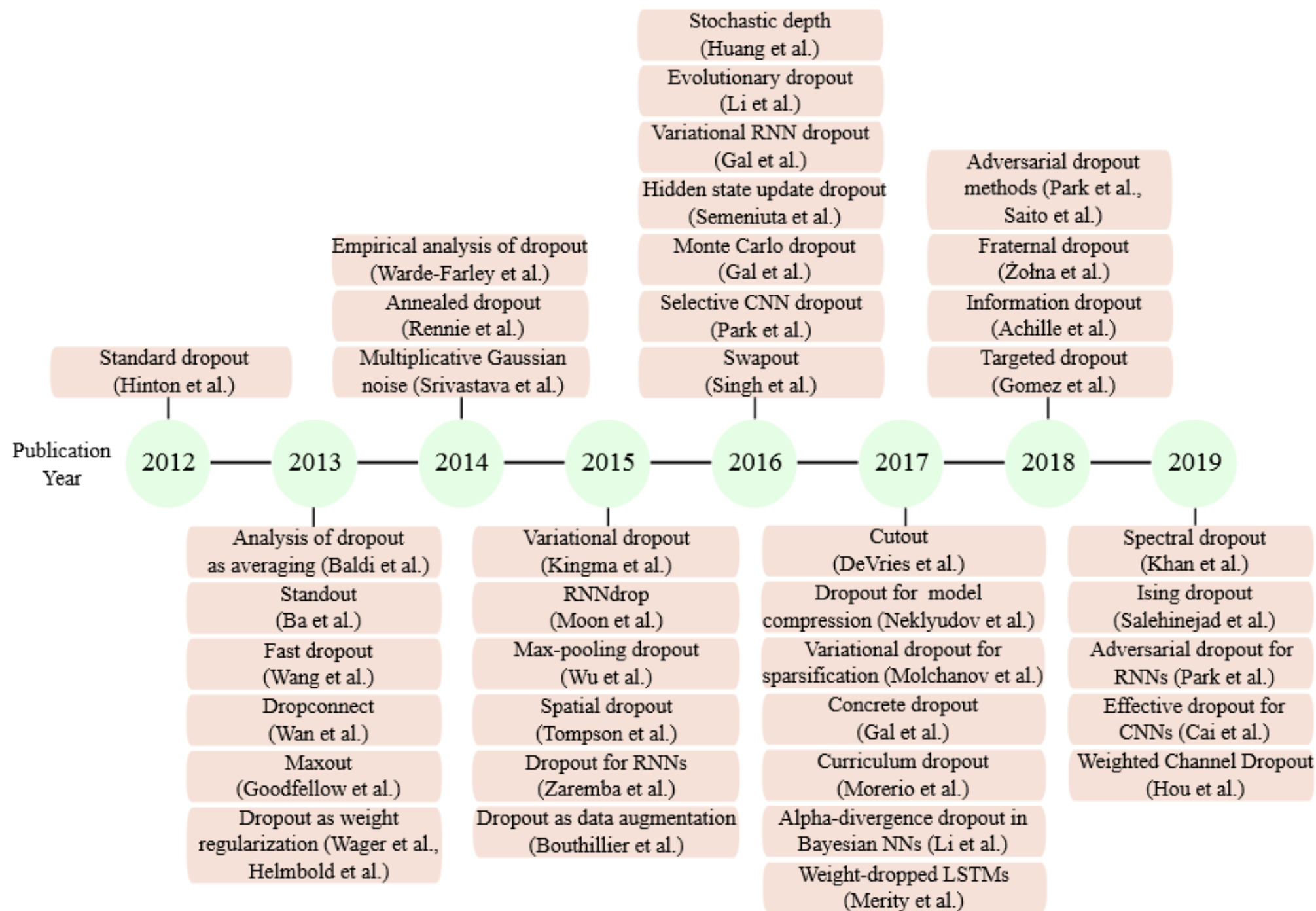
Figure 1. (a): An example model layout for a single DropConnect layer. After running feature extractor $g()$ on input x , a random instantiation of the mask M (e.g. (b)), masks out the weight matrix W . The masked weights are multiplied with this feature vector to produce u which is the input to an activation function a and a softmax layer s . For comparison, (c) shows an effective weight mask for elements that Dropout uses when applied to the previous layer's output (red columns) and this layer's output (green rows). Note the lack of structure in (b) compared to (c).

Gaussian Dropout



Fast Dropout / Variational Dropout / Concrete Dropout

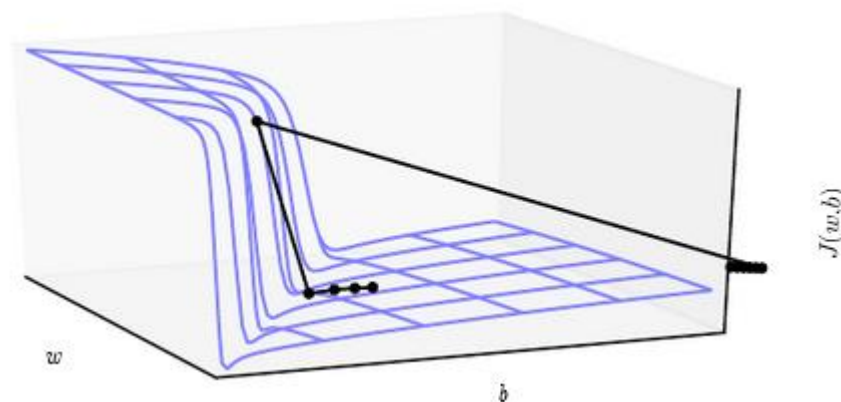
Все эти приёмы неявной регуляризации –
ухудшаем оптимизацию на train, но уменьшается переобучение



«Survey of Dropout Methods for Deep Neural Networks» <https://arxiv.org/pdf/1904.13310.pdf>

Обрезка градиентов (Gradient clipping)

Если «попали на утёс», может слишком уйти от минимума



Выход – укоротить вектор градиента
(направление сохраняется)

$$g = \frac{\partial L}{\partial w}$$

$$g^{\text{new}} = \frac{\min(\theta, \|g\|)}{\|g\|} g$$

Работает пока дисперсия градиента маленькая...
(если градиент ~ колоколообразно распределён, то ясно как ограничить его,
но если колокол начинает «расширяться» ...)

Pascanu, Mikolov, Bengio для RNN

Минутка кода

```
optimizer.zero_grad()  
loss, hidden = model(data, hidden, targets)  
loss.backward() # после этого  
  
torch.nn.utils.clip_grad_norm(model.parameters(), max_norm)  
optimizer.step() # до этого
```

Батч-нормализация (Batch normalization)

– метод адаптивной перепараметризации

Проблема:

как изменять параметры, при условии, что вся остальная сеть **не меняется**
трудно предсказать, насколько изменится какое-то значение
(оно зависит от всех предыдущих в суперпозиции)

Covariate shift – изменение распределений входов во время обучения
Надо уменьшить это изменение в скрытых слоях!

Ioffe and Szegedy, 2015 <https://arxiv.org/abs/1502.03167>

Батч-нормализация (Batch normalization)

$$\{x_i\}_{i=1}^m$$

минибатч

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

среднее по мини-батчу

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

дисперсия по мини-батчу

$$x_i^{\text{new}} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

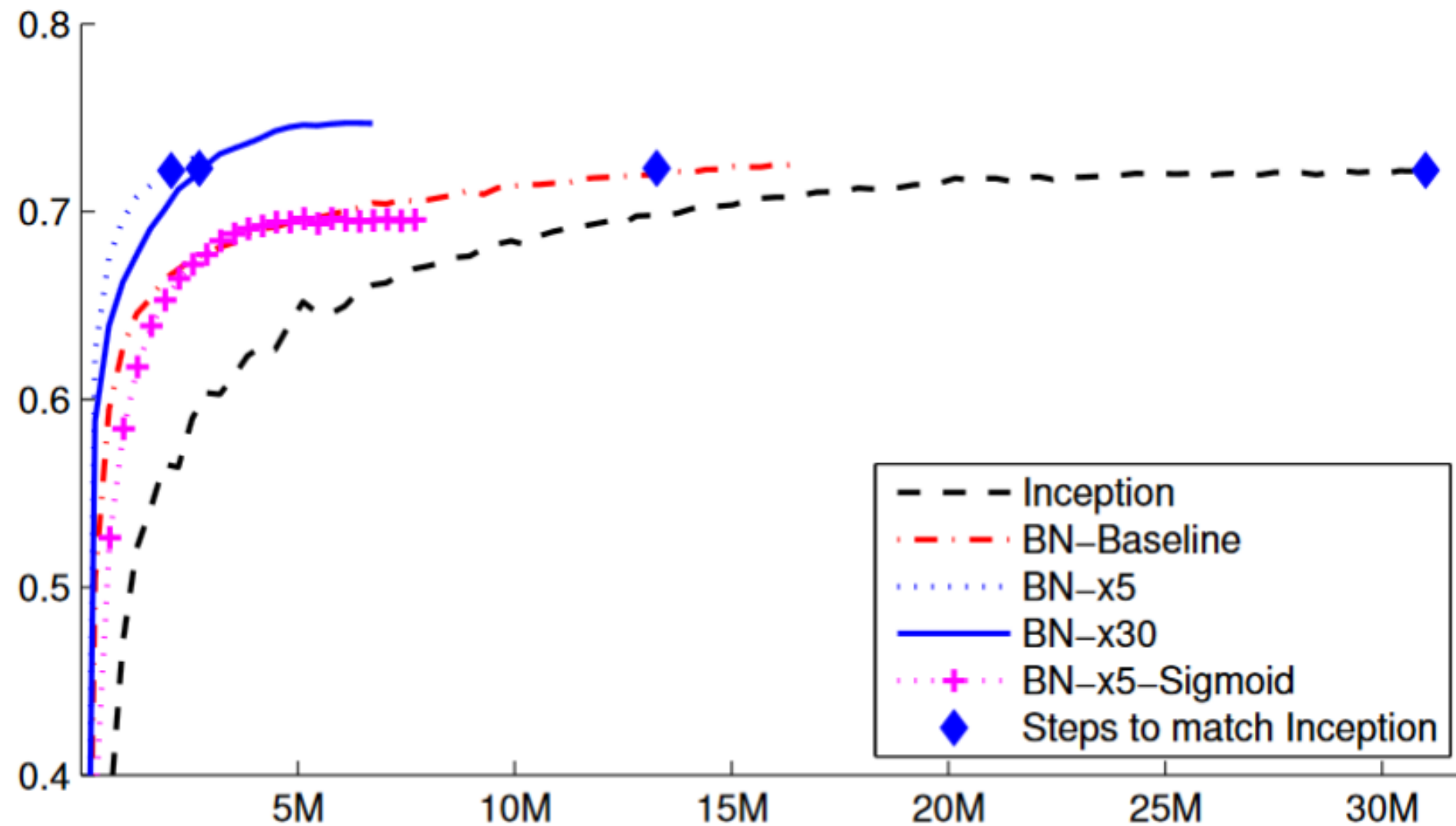
нормировка

$$y_i = \gamma x_i^{\text{new}} + \beta$$

растяжение и сдвиг

Надо определить параметры γ и β

Батч-нормализация (Batch normalization)



Обучение Inception с / без батч-нормализацией
+ варианты с увеличением темпа обучения

Батч-нормализация (Batch normalization)

При обучении:

среднее и дисперсия
по мини-батчу

При тесте:

среднее и дисперсия –

- по обучению
- усреднение значений, что были во время обучения
- экспоненциальное среднее $=*$

здесь тоже нужно переключение режимов

нормализация перед входом в каждый слой (иногда до активации, иногда после)

если Linear + BN,
то линейный слой делать без смещения

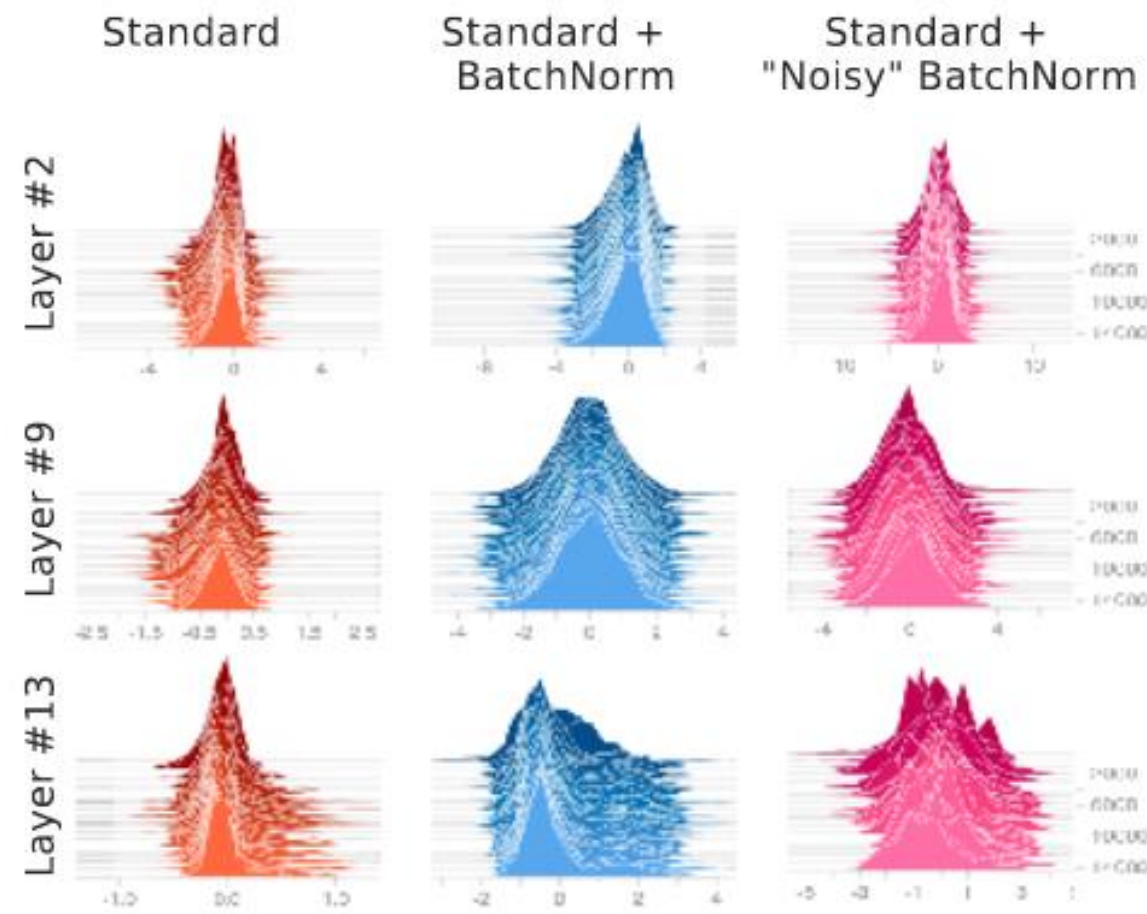
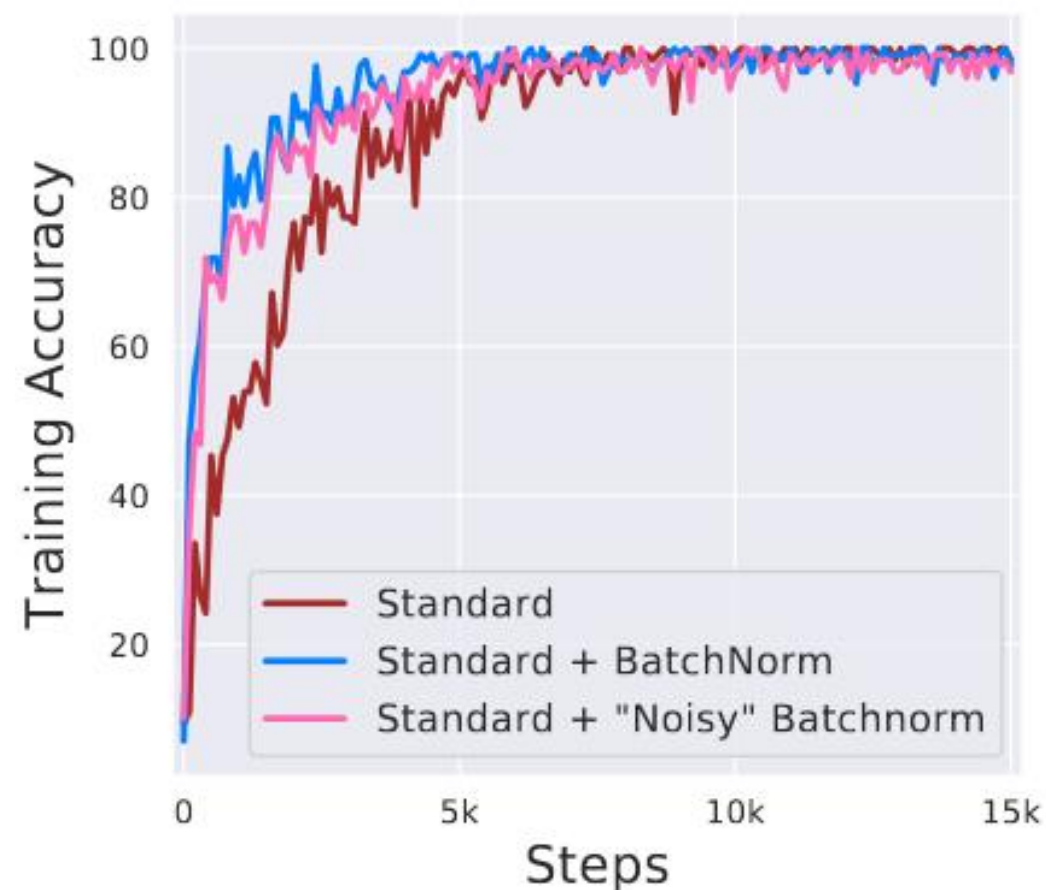
Батч-нормализация (Batch normalization)

- можно увеличить скорость обучения
 - можно убрать dropout
 - можно уменьшить регуляризацию
и так есть шум из-за случайных мини-батчей
- можно использовать более глубокие сети
- меньше чувствительности к инициализации / выбору активации
- когда сеть обучена и не будет меняться м.б. BN можно устранить,
например свёртка + BN = свёртка

Удалось убрать LRN (Local Response Normalization),
но надо тщательнее перемешивать обучение

Батч-нормализация (Batch normalization)

– обоснование эффективности – открытая проблема



дело не в изменении распределения, а в сглаживании функции

S. Santurkar, D. Tsipras, A. Ilyas, A. Madry «How Does Batch Normalization Help Optimization?» 2018 // <https://arxiv.org/abs/1805.11604>

Батч-нормализация (Batch normalization)

DropOut и BatchNorm реализуются как отдельные слои:

```
self.bn1 = nn.BatchNorm1d (num_features=n1, eps=1e-05,  
                           momentum=0.1, affine=True,  
                           track_running_stats=True)  
self.do1 = nn.Dropout(p=0.1, inplace=False)  
self.ln1 = nn.Linear(n1, n2)  
self.ph1 = nn.ReLU()
```

Batch Normalization (BN) – если размер батча маленький, то всё плохо...

мало статистики

а иногда необходимы / желательны маленькие батчи (bs=1)

Хотя есть мнение, что маленький батч сильнее регуляризует!

дальше ещё варианты нормализации

Минутка кода

```
from torch import nn
H = torch.arange(1, 17).reshape(4, 4).float()
```

```
drop = nn.Dropout(p=0.5) # Dropout
```

```
print(H, drop(H))
```

```
tensor([[ 1.,  2.,  3.,  4.],
        [ 5.,  6.,  7.,  8.],
        [ 9., 10., 11., 12.],
        [13., 14., 15., 16.]])
```

```
tensor([[ 0.,  4.,  0.,  0.],
        [ 0.,  0., 14., 16.],
        [18.,  0., 22., 24.],
        [ 0., 28.,  0.,  0.]])
```

```
bn = nn.BatchNorm1d(4, affine=False) # BN1D
```

```
print(bn(H))
```

```
tensor([[[-1.341, -1.341, -1.341, -1.341],
         [-0.447, -0.447, -0.447, -0.447],
         [ 0.447,  0.447,  0.447,  0.447],
         [ 1.341,  1.341,  1.341,  1.341]])])
```

Ещё нормализации

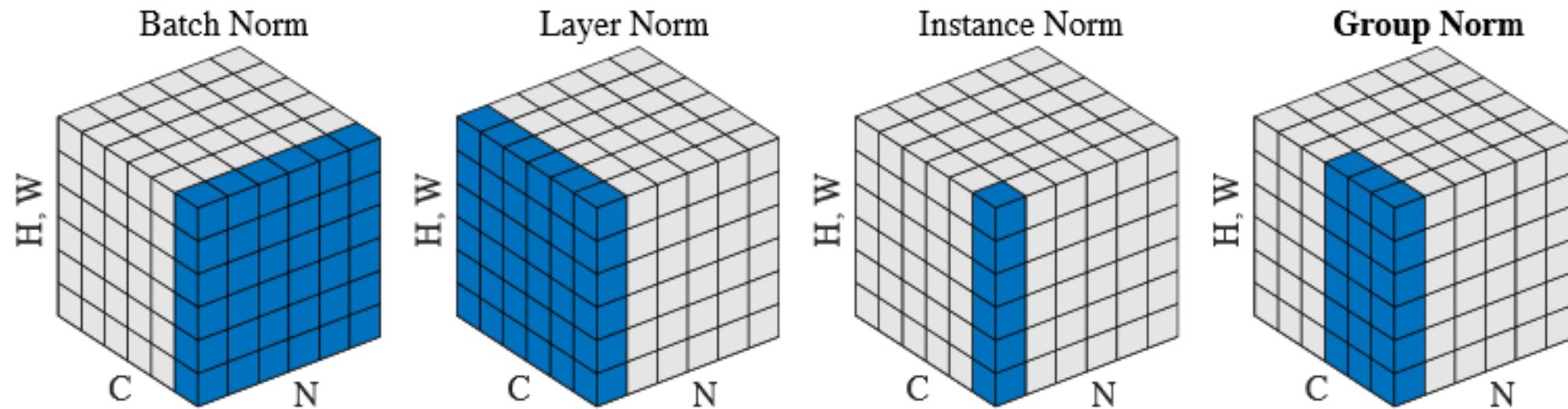


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

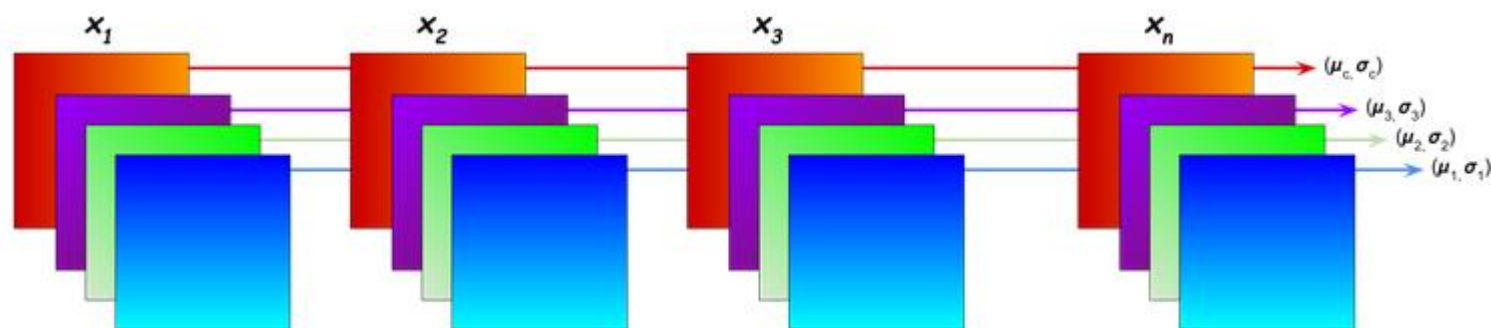
Layer Normalization – Ba, Kiros, and Hinton, «Layer Normalization», arXiv 2016

Instance Normalization – Ulyanov et al «Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis», CVPR 2017

Group Normalization – Wu and He, «Group Normalization» arXiv 2018

<https://becominghuman.ai/all-about-normalization-6ea79e70894b>

BN – средние и дисперсии по каналам (признакам)

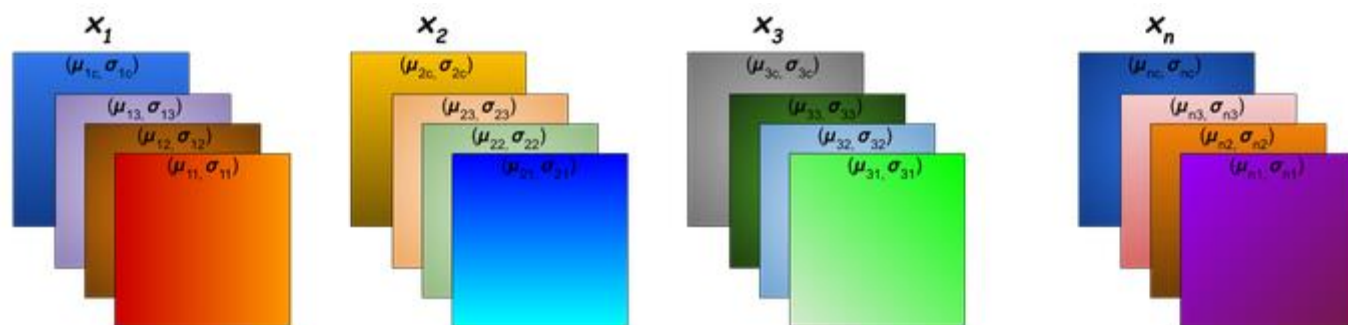


$$\mu_c = \frac{1}{NHW} \sum_{i=1}^N \sum_{j=1}^H \sum_{k=1}^W x_{icjk}$$

$$\sigma_c^2 = \frac{1}{NHW} \sum_{i=1}^N \sum_{j=1}^H \sum_{k=1}^W (x_{icjk} - \mu_c)^2$$

$$\hat{x} = \frac{x - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

IN – для пары (объект, канал) (лучше ~style transfer)

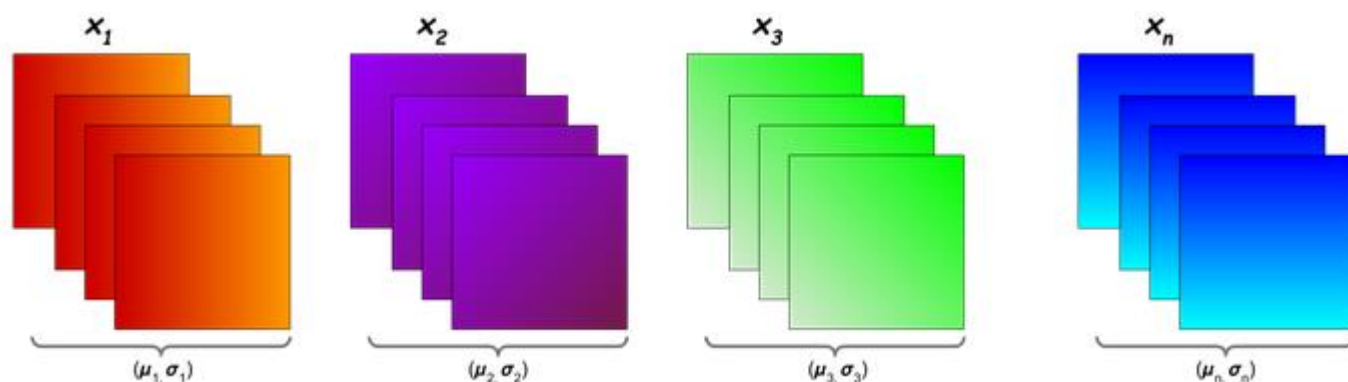


$$\mu_{nc} = \frac{1}{HW} \sum_{j=1}^H \sum_{k=1}^W x_{ncjk}$$

$$\sigma_{nc}^2 = \frac{1}{HW} \sum_{j=1}^H \sum_{k=1}^W (x_{ncjk} - \mu_{nc})^2$$

$$\hat{x} = \frac{x - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}}$$

LN – для объекта (лучше ~RNN)



$$\mu_n = \frac{1}{CHW} \sum_{i=1}^C \sum_{j=1}^H \sum_{k=1}^W x_{nijk}$$

$$\sigma_n^2 = \frac{1}{CHW} \sum_{i=1}^C \sum_{j=1}^H \sum_{k=1}^W (x_{nijk} - \mu_n)^2$$

$$\hat{x} = \frac{x - \mu_n}{\sqrt{\sigma_n^2 + \epsilon}}$$

Ещё нормализации

Batch renormalization = BN + MA (4 inference)

Batch-Instance Normalization ~ ЛК batch normalization + instance normalization
(коэффициент выучивается)

Минутка кода: если бы BN с нуля...

```
def batch_norm(X, gamma, beta, moving_mean, moving_var, eps, momentum):
    if not torch.is_grad_enabled():
        # prediction mode - directly use the mean and variance obtained by moving average
        X_hat = (X - moving_mean) / torch.sqrt(moving_var + eps)
    else:
        assert len(X.shape) in (2, 4)
        if len(X.shape) == 2:
            # fully-connected layer => calculate the mean and variance on the feature dimension
            mean = X.mean(dim=0)
            var = ((X - mean) ** 2).mean(dim=0)
        else:
            # two-dimensional convolutional layer => mean and variance on axis=1 (channel)
            mean = X.mean(dim=(0, 2, 3), keepdim=True)
            var = ((X - mean) ** 2).mean(dim=(0, 2, 3), keepdim=True)
        # In training mode, the current mean and variance are used for the standardization
        X_hat = (X - mean) / torch.sqrt(var + eps)
        # Update the mean and variance using moving average
        moving_mean = momentum * moving_mean + (1.0 - momentum) * mean
        moving_var = momentum * moving_var + (1.0 - momentum) * var
    Y = gamma * X_hat + beta # Scale and shift
    return Y, moving_mean.data, moving_var.data
```

Минутка кода: если бы BN с нуля...

```
class BatchNorm(nn.Module):
    # `num_features`: # outputs for a fully-connected layer or # output channels for a convolutional layer.
    # `num_dims`: 2 for a fully-connected layer and 4 for a convolutional layer
    def __init__(self, num_features, num_dims):
        super().__init__()
        if num_dims == 2:
            shape = (1, num_features)
        else:
            shape = (1, num_features, 1, 1)
        # The scale parameter and the shift parameter (model parameters)
        self.gamma = nn.Parameter(torch.ones(shape))
        self.beta = nn.Parameter(torch.zeros(shape))
        # The variables that are not model parameters are initialized to 0 and 1
        self.moving_mean = torch.zeros(shape)
        self.moving_var = torch.ones(shape)

    def forward(self, X):
        # copy `moving_mean` and `moving_var` to the device where `X` is located
        if self.moving_mean.device != X.device:
            self.moving_mean = self.moving_mean.to(X.device)
            self.moving_var = self.moving_var.to(X.device)
        # Save the updated `moving_mean` and `moving_var`
        Y, self.moving_mean, self.moving_var = batch_norm(X, self.gamma, self.beta, self.moving_mean,
                                                            self.moving_var, eps=1e-5, momentum=0.9)
        return Y
```

https://d2l.ai/chapter_convolutional-modern/batch-norm.html

Итог

Много трюков... их число увеличивается
Важны также не отдельные трюки, а комбинации

Иногда «старые трюки» меняются
т.к. не работают в комбинации с новыми,
появляются новые архитектуры,
увеличиваются размеры моделей и датасетов

список ссылок и литературы будет на следующей лекции